MAKSDEV STUDIO

বাংলা সংস্করণ / PYTHON 3

পাইথন

প্রথম লাইন থেকে বাস্তব প্রজেক্ট

যুক্তি বুঝুন। কোড লিখুন।
কাজে লাগে এমন কিছু বানান।

< 01 → 10 → ∞ />

৩১টি অধ্যায় ও বিভাগ · ৪টি প্রজেক্ট
১০০টি challenge · ৪৭টি runnable example

Md. Makshedul Islam

Student, EXIM Bank Agricultural University Bangladesh.

শেখার শুরু

পাইথন দিয়ে কাজ শুরু করার আগে পুরো ভাষা মুখস্থ করতে হয় না। কয়েকটি ধারণা ভালোভাবে বুঝলেই শুরু করা যায়: তথ্য প্রকাশ করে value, হিসাব করে expression, পথ বেছে নেয় condition, একই কাজ বারবার করে loop, আর একটি কাজকে নাম দিয়ে পুনরায় ব্যবহার করতে সাহায্য করে function।

এই বইয়ে আমরা সমস্যার ভাষা থেকে কোডের ভাষায় যাব। আগে ঠিক করব কী তথ্য আছে ও কী ফল চাই। তারপর যুক্তি সাজাব, কোড চালানোর আগে ফল অনুমান করব, এবং ফল না মিললে কারণ খুঁজব। অন্যের উত্তর টাইপ করাই লক্ষ্য নয়; উত্তরটি কেন কাজ করে এবং কোন পরিস্থিতিতে কাজ করবে না, তা বুঝতে হবে।

বইটি কীভাবে পড়বেন

1. কোড দেখার আগে সমস্যাটি পড়ুন।
2. input ও কাঙ্ক্ষিত output লিখুন।
3. ছোট একটি উদাহরণ হাতে সমাধান করুন।
4. কোডের প্রতিটি ধাপে কোন মান বদলাচ্ছে তা অনুসরণ করুন।
5. কোড চালিয়ে নিজের অনুমানের সঙ্গে মিলিয়ে দেখুন।
6. একটি মান বা শর্ত বদলে আবার অনুমান করুন।
7. সমাধান দেখার আগে অনুশীলন নিজে চেষ্টা করুন।

একটি খাতায় চারটি বিষয় রাখুন: **আমার পরিকল্পনা, আমার অনুমান, পাওয়া ফল, ফলটির ব্যাখ্যা**। অনুমান ভুল হওয়া শেখার গুরুত্বপূর্ণ অংশ। ভুল ঢাকার বদলে তাকে প্রশ্ন করুন।

ভাষা ও সংস্করণ

ব্যাখ্যা বাংলায়; code, identifier, terminal command এবং console output ইংরেজিতে রাখা হয়েছে। এতে অফিসিয়াল documentation পড়তে ও বাস্তব project-এ কাজ করতে সুবিধা হবে। গুরুত্বপূর্ণ ইংরেজি শব্দের অর্থ ব্যাখ্যার মধ্যেই পাবেন।

Python 3.10 বা নতুন সংস্করণ ব্যবহার করুন। `match` ও `int | str`-এর মতো কিছু syntax-এর জন্য অন্তত 3.10 প্রয়োজন। python.org থেকে সমর্থিত Python 3 সংস্করণ নিন। কোনো তৃতীয় পক্ষের package বা online service চিরকাল একই রকম থাকবে—এমন ধরে নেবেন না।

বইয়ের ৪৭টি Python code block আলাদা `.py` ফাইল হিসেবেও দেওয়া আছে। কয়েকটি উদাহরণ input চায় বা অনুশীলনের ফাইল তৈরি করে। তাই নিজের একটি practice folder-এ চালান। চারটি বড় project-এর সহজ নামের কপিও আছে।

সূচিপত্র

ভিত্তি থেকে নিজের প্রজেক্ট

1 · প্রোগ্রাম মানে নির্ভুল নির্দেশনা 4
2 · Python সেটআপ, editor ও terminal 6
3 · Value, variable ও data type 9
4 · Input, output ও type conversion 11
5 · Operator ও expression 13
6 · Condition ও Boolean logic 15
7 · While loop দিয়ে পুনরাবৃত্তি 17
8 · For loop, range ও enumerate 19
9 · Nested loop, tracing ও pattern 20
10 · String ও text processing 22
11 · List ও mutable object 24
12 · Tuple ও unpacking 26
13 · Dictionary ও record 27
14 · Set ও membership 29
15 · Function, argument ও return 30
16 · Scope, recursion ও সমস্যা ভাগ করা 32
17 · Module, package ও virtual environment 34
18 · File, path, CSV ও JSON 36
19 · Exception, validation ও debugging 38
20 · Test ও program design 40
21 · Class ও object-oriented চিন্তা 42
22 · Inheritance, property ও special method 44
23 · Comprehension, iterator ও functional tool 46
24 · Type hint, match ও walrus 48
25 · প্রজেক্ট: নির্ভরযোগ্য calculator 50
26 · প্রজেক্ট: quiz ও score report 52
27 · প্রজেক্ট: JSON-এ খরচের হিসাব 54
28 · প্রজেক্ট: assistant ও reply draft 58
29 · একশটি practice challenge 60
30 · নির্বাচিত সমস্যার যুক্তিসহ সমাধান 64
31 · Quick reference ও পরবর্তী পথ 71

1 · প্রোগ্রাম মানে নির্ভুল নির্দেশনা

সমস্যা থেকে পরিকল্পনা

একটি দোকানে প্রতি খাতার দাম ৪৫ টাকা। তিনটি খাতার মোট দাম কত? মানুষের কাছে হিসাবটি সহজ হলেও কম্পিউটারকে তথ্য ও নিয়ম—দুটিই জানাতে হয়।

Input: খাতার সংখ্যা ও প্রতিটির দাম। **Process:** সংখ্যা দিয়ে একক দাম গুণ করা। **Output:** মোট দাম দেখানো। এই ধাপে ধাপে সমাধানের পদ্ধতিই algorithm বা অ্যালগরিদম। এটি কোনো নির্দিষ্ট programming language-এর সম্পত্তি নয়; বাংলায় লিখেও algorithm বোঝানো যায়।



তথ্য প্রবেশ করে, নিয়ম প্রয়োগ হয়, তারপর ফল পাওয়া যায়।

```
quantity = 3
price = 45
total = quantity * price
print(f"Total: {total} taka")
```

প্রত্যাশিত output

```
Total: 135 taka
```

লাইন ধরে বুঝি

- `quantity = 3` নামটিকে integer value 3-এর সঙ্গে যুক্ত করে।
- `price = 45` একক দাম রাখে।
- `quantity * price` আগে হিসাব হয়; ফল 135 এরপর `total` নামের সঙ্গে যুক্ত হয়।
- `print()` ফলটি পর্দায় দেখায়। এটি টাকা আদায় করে না বা বাস্তব কোনো যন্ত্র চালায় না।

ধাপের প্রবাহ: শুরু → সংখ্যা ও দাম নিন → মোট হিসাব করুন → ফল দেখান → শেষ। Flowchart-এ শুরু/শেষের জন্য oval, input/output-এর জন্য parallelogram, কাজের জন্য rectangle এবং সিদ্ধান্তের জন্য diamond ব্যবহৃত হয়। Arrow নির্দেশ করে কোন ধাপের পর কোন ধাপ আসবে।

ভুল এড়ান

কোডের আগে সমস্যার অর্থ ঠিক করুন। 135 টাকা আর 135 খাতা একই তথ্য নয়; unit গুরুত্বপূর্ণ। `print("Open the door")` শুধু বাক্যটি দেখায়, দরজা খুলে দেয় না।

অনুশীলন

আয়তক্ষেত্রের ক্ষেত্রফল বের করার input, formula ও output লিখুন। এমন একটি উদাহরণ বাছুন যার উত্তর হাতে যাচাই করতে পারেন। তারপর কোড লিখুন।

2 · Python সেটআপ, editor ও terminal

কোথায় কী লিখবেন

Terminal operating system-এর command চালায়। Python interpreter Python statement ও expression চালায়। Terminal prompt-এর `$` বা `>` এবং Python REPL-এর `>>>` চিহ্ন সাধারণত নিজে টাইপ করতে হয় না।

Windows-এ `py`, macOS/Linux-এ `python3` পাওয়া যেতে পারে। এখানে `python` লেখা হয়েছে। আপনার কম্পিউটারে যে command সঠিক Python 3 চালায়, সেটি ব্যবহার করুন।

```
python --version
python -m pip --version
```

Python পাওয়া না গেলে ইনস্টল করুন এবং installer-এর PATH-সংক্রান্ত নির্দেশনা অনুসরণ করুন। Linux-এর system Python বদলে না দিয়ে distribution-এর নির্দেশনা ব্যবহার করুন। VS Code সুবিধাজনক, কিন্তু প্রথম উদাহরণের জন্য সাধারণ text editor-ও যথেষ্ট।

প্রথম ফাইল

`python-work` নামে folder বানান। নিচের code `hello.py` নামে রাখুন; ভুল করে `hello.py.txt` নয়।

```python
```

A comment explains intent; it is not executed.

name = "Python learner" print(f"Hello, {name}!") print("A small program is still a real program.")

> সেই `folder-এ terminal খুলে চালান:`
>
> ```sh
> python hello.py

ফাইল save করা আর run করা আলাদা কাজ। পরিবর্তন save না করলে পুরোনো code-ই চলতে পারে। `#` দিয়ে comment লেখা হয়। Comment কী করছি তার চেয়ে কেন করছি তা ব্যাখ্যা করলে বেশি উপকার হয়।

REPL কী

ফাইলের নাম ছাড়া `python` চালালে interactive REPL খুলতে পারে। সেখানে `2 + 3` লিখলে ফল দেখা যায়। কিন্তু script-এ শুধু expression লিখলে ফল স্বয়ংক্রিয়ভাবে print হয় না; `print(2 + 3)` লিখতে হয়। `exit()` দিয়ে বের হতে পারেন।

Indentation সাজসজ্জা নয়

Python-এ indentation দিয়ে block বোঝানো হয়। প্রতি level-এ চারটি space প্রচলিত নিয়ম। Tab ও space মেশাবেন না। Expression-এর ভেতরে কিছু extra space গ্রহণযোগ্য হলেও indentation ও কিছু newline-এর অর্থ আছে।

সমস্যা হলে

লক্ষণ	কী দেখবেন
File not found	terminal কোন folder-এ, filename ঠিক কি না
SyntaxError	দেখানো line ও তার আগের line-এ bracket/quote/colon
Output নেই	`print()` আছে কি, function call করা হয়েছে কি

লক্ষণ	কী দেখবেন
Editor ও terminal-এ আলাদা ফল	একই interpreter ও একই file চালাচ্ছেন কি

অনুশীলন

তিন লাইনে নিজের নাম, automate করতে চান এমন একটি কাজ এবং `12 * 12`-এর ফল দেখান। Code বদলানোর পর save করে আবার চালান।

3 · Value, variable ও data type

সহজ mental model

Value হলো তথ্যের একটি মান। Type নির্ধারণ করে তার সঙ্গে কোন operation অর্থপূর্ণ। Variable হলো কোনো object-এর সঙ্গে যুক্ত নাম; চিরস্থায়ীভাবে নির্দিষ্ট type-এর বাক্স নয়।

Type	উদাহরণ	সাধারণ ব্যবহার
`int`	`42`	পূর্ণসংখ্যা, গণনা
`float`	`2.5`	আনুমানিক দশমিক হিসাব
`str`	`"Dhaka"`	লেখা
`bool`	`True`	সত্য/মিথ্যা সিদ্ধান্ত
`NoneType`	`None`	মান অনুপস্থিত বা এখনো দেওয়া হয়নি

```
students = 28
average = 76.5
course = "Python"
is_open = True
result = None
print(type(students).__name__)
print(course, is_open, result)
students = students + 1
print(students)
```

```
int
Python True None
29
```

`students = students + 1`-এ ডান পাশ আগে হিসাব হয়। পুরোনো 28-এর সঙ্গে 1 যোগ করে নতুন 29-এর সঙ্গে নামটি যুক্ত হয়। Assignment কোনো বীজগণিতের সমীকরণ নয়।

ভালো নাম বাছুন

`student_count`, `unit_price`, `is_valid` অর্থ পরিষ্কার করে। নাম case-sensitive: `total` ও `Total` আলাদা। সংখ্যা দিয়ে নাম শুরু করা যায় না এবং `class`-এর মতো keyword নাম হিসেবে ব্যবহার করা যায় না। `sum`, `list`, `str`, `input`-এর মতো built-in নাম নিজের variable দিয়ে ঢেকে ফেলবেন না।

Equality আর identity

`==` দিয়ে মান সমান কি না দেখা হয়। `is` দেখে দুটি reference একই object-কে নির্দেশ করছে কি না। `None` পরীক্ষা করতে `value is None` লিখুন। সাধারণ string বা number-এর মান মেলাতে `is` ব্যবহার করবেন না।

Float পুরোপুরি exact নয়

Binary floating-point-এ সব দশমিক ভগ্নাংশ নিখুঁতভাবে রাখা যায় না। তাই `0.1 + 0.2` ও `0.3` সরাসরি সমান নাও হতে পারে। Measurement তুলনায় tolerance দরকার হতে পারে। অর্থের project-এ আমরা integer পয়সা ব্যবহার করব, বারবার float যোগ করব না।

অনুশীলন

ফোন নম্বর, শিক্ষার্থীর নাম, উপস্থিতির সংখ্যা ও pass/fail-এর জন্য type বাছুন। ফোন নম্বর সাধারণত string: এর ওপর যোগ-বিয়োগ করি না এবং শুরুর শূন্য গুরুত্বপূর্ণ হতে পারে।

4 · Input, output ও type conversion

সমস্যা

নাম ও বয়স নিয়ে পরের বছর বয়স কত হবে তা দেখাতে হবে। `input()` সবসময় string ফেরত দেয়। ব্যবহারকারী `19` লিখলেও সেটি আগে text।

```
name = input("Name: ").strip()
age_text = input("Age in whole years: ")
age = int(age_text)
print(f"Hello, {name}. Next year you will be {age + 1}.")
```

Input হিসেবে `Amina` ও `19` দিলে শেষ line হবে:

```
Hello, Amina. Next year you will be 20.
```

যুক্তির চার ধাপ

1. নাম নিন; `.strip()` দিয়ে দুই পাশের whitespace সরান।
2. বয়স text হিসেবে নিন।
3. `int()` দিয়ে পূর্ণসংখ্যায় রূপান্তর করুন।
4. 1 যোগ করে f-string দিয়ে ফল দেখান।

`nineteen` দিলে `ValueError` হবে। এটি অনুমেয় input সমস্যা। ১৯ নম্বর অধ্যায়ে user-কে বুঝিয়ে আবার input নেওয়া শিখব।

Conversion-এর সীমা

`int("12")` চলে, কিন্তু `int("12.5")` চলে না। `int(12.9)` দেয় 12; এটি nearest integer-এ round করে না, শূন্যের দিকে ভগ্নাংশ বাদ দেয়। `str(12)` text বানায়। `float("12.5")` আনুমানিক দশমিক সংখ্যা বানায়।

`print(a, b)` সাধারণত মাঝখানে space দেয়। `sep=" | "` separator বদলায়; `end=""` শেষের newline বন্ধ করে। `f"{price:.2f}"` দেখানোর সময় দুই ঘর দশমিক রাখে; ভেতরের arithmetic exact হয়ে যায় না।

প্রচলিত ভুল

- `input() + 1`: string-এর সঙ্গে integer যোগ করা হচ্ছে।
- `"10" + "5"`: ফল `"105"`, কারণ string জোড়া লাগছে।
- `bool("False")`: ফল `True`, কারণ string খালি নয়।
- `.strip()` নামের ভেতরের সব space সরাবে ধরে নেওয়া; এটি প্রান্তের whitespace সরায়।

অনুশীলন

দৈর্ঘ্য ও প্রস্থ `float()` দিয়ে নিয়ে ক্ষেত্রফল দেখান। খালি text, ঋণাত্মক dimension ও অক্ষর লিখলে কী হওয়া উচিত তা আগে ঠিক করুন। পরে validation যোগ করুন।

5 · Operator ও expression

Operator	কাজ	উদাহরণের ফল
`+`, `-`, `*`	যোগ, বিয়োগ, গুণ	`7 * 3` → `21`
`/`	true division	`7 / 2` → `3.5`
`//`	নিচের পূর্ণসংখ্যার দিকে ভাগফল	`7 // 2` → `3`
`%`	remainder	`7 % 2` → `1`
`**`	ঘাত	`2 ** 3` → `8`
`==`, `!=`	সমান, অসমান	`3 != 4` → `True`
`<`, `<=`, `>`, `>=`	তুলনা	`5 >= 5` → `True`

সমস্যা: মিনিট থেকে ঘণ্টা

```
total_minutes = 135
hours = total_minutes // 60
minutes = total_minutes % 60
print(f"{hours} hours, {minutes} minutes")
print(-7 // 2)
print(2 + 3 * 4)
print((2 + 3) * 4)
```

```
2 hours, 15 minutes
-4
14
20
```

১৩৫ মিনিটে কয়টি পূর্ণ ৬০ মিনিট আছে তা `//` বলে। বাকি মিনিট `%` দেয়। Negative number-এ floor division শূন্যের দিকে নয়, ঋণাত্মক অসীমের দিকে নিচে নামে। তাই `-7 // 2` হলো -4।

হিসাবের ক্রম

গুণ আগে হওয়ায় `2 + 3 * 4` হলো 14। Parentheses দিলে `(2 + 3) * 4` হলো 20। জটিল precedence মুখস্থ দেখানোর বদলে স্পষ্ট parentheses দিন। `2 ** 3 ** 2` মানে `2 ** (3 ** 2)`।

`score += 5` একটি সংক্ষিপ্ত update। Number-এর ক্ষেত্রে নাম নতুন ফলের সঙ্গে যুক্ত হয়। Mutable collection-এর কিছু augmented operation একই object বদলাতে পারে; সব `+=`-এর reference behavior এক ধরে নেবেন না।

Bitwise operation

`&`, `|`, `^`, `~`, `<<`, `>>` integer-এর bit নিয়ে কাজ করে। এগুলো Boolean `and`, `or`, `not`-এর সাধারণ বিকল্প নয়। Mask বা binary protocol নিয়ে কাজের আগে এই অংশ বিস্তারিত শেখা বাধ্যতামূলক নয়।

অনুশীলন

৯৫টি জিনিস ১২টি করে বাক্সে ভরলে কয়টি পূর্ণ বাক্স হবে, কয়টি বাকি থাকবে? Run করার আগে উত্তর লিখুন।

6 · Condition ও Boolean logic

সমস্যা: নম্বর থেকে grade

প্রথমে নিশ্চিত হতে হবে নম্বর 0 থেকে 100-এর মধ্যে। এরপর বড় threshold থেকে ছোট threshold পরীক্ষা করব।

```
mark = 76
if not 0 <= mark <= 100:
    grade = "Invalid"
elif mark >= 80:
    grade = "A"
elif mark >= 70:
    grade = "B"
elif mark >= 60:
    grade = "C"
elif mark >= 50:
    grade = "D"
else:
    grade = "F"
print(grade)
```

Output: `B`

`if`/`elif` chain-এর প্রথম সত্য branch-ই চলে। আগে `mark >= 50` দেখলে 90-ও নিচের grade পেতে পারে। তাই শর্ত সঠিক হওয়ার পাশাপাশি ক্রমও সঠিক হওয়া দরকার।

Flow: শুরু → range সঠিক? না হলে invalid দেখিয়ে শেষ। ঠিক হলে বড় থেকে ছোট grade threshold পরীক্ষা → grade দেখান → শেষ।

Truth table

a	b	a and b	a or b
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

দুটি operand-ই Boolean হলে টেবিলটি সরাসরি প্রযোজ্য। সাধারণভাবে Python-এর `and` ও `or` operand-এর একটি ফেরত দেয়; সবসময় Boolean নয়। যেমন `name or "Guest"` non-empty নাম অথবা fallback text দেয়। `not` Boolean ফেরত দেয়।

Short-circuit

ফল জানা হয়ে গেলে Python বাকি expression চালায় না। `denominator != 0 and numerator / denominator > 2`-এ denominator শূন্য হলে ভাগ করা হয় না। উল্টো ক্রমে লিখলে এই সুরক্ষা থাকবে না।

Zero, `None` ও empty container falsey। বেশিরভাগ অন্য object truthy। `if items:` দিয়ে non-empty কি না দেখা যায়। শুধু space থাকা string-ও non-empty; প্রয়োজন হলে আগে strip করুন।

ভুল চিনুন

`if answer == "yes" or "y":` ভুল, কারণ `"y"` নিজেই truthy। লিখুন `answer in {"yes", "y"}`। তুলনায় `==`, assignment-এ `=`। Grade পরীক্ষা করতে শুধু 76 নয়, 49, 50, 79, 80 ও 100-এর মতো boundary নিন।

অনুশীলন

Leap year নির্ধারণ করুন: 400 দিয়ে বিভাজ্য, অথবা 4 দিয়ে বিভাজ্য কিন্তু 100 দিয়ে নয়। 1900, 2000 ও 2024 দিয়ে যাচাই করুন।

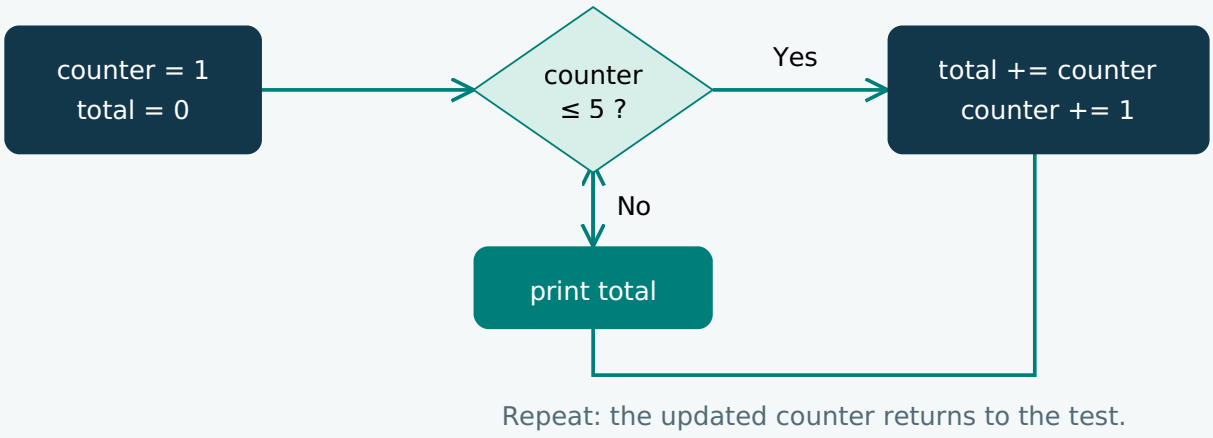
7 · While loop দিয়ে পুনরাবৃত্তি

সমস্যা: 1 থেকে 5-এর যোগফল

একটি counter বলবে এখন কোন সংখ্যা যোগ হচ্ছে। আরেকটি variable running total রাখবে।

```
counter = 1
total = 0
while counter <= 5:
    total += counter
    counter += 1
print(total)
```

Output: 15



প্রতি iteration-এর আগে condition পরীক্ষা; counter update হয়ে আবার test-এ ফেরে।

Trace table

Body শুরুর counter	আগের total	যোগের পর total	পরের counter
1	0	1	2
2	1	3	3
3	3	6	4
4	6	10	5

Body শুরুর counter	আগের total	যোগের পর total	পরের counter
5	10	15	6

এরপর `6 <= 5` মিথ্যা, তাই body আর চলে না। Counter বাড়ানোর line বাদ দিলে condition বদলাবে না এবং loop অনন্ত চলতে পারে।

Loop-এর তিন প্রশ্ন

শুরুতে কী সত্য? প্রতি iteration-এ কী বদলাচ্ছে? কেন একসময় থামবে? এখানে প্রতিবার body-র আগে total হলো counter-এর আগ পর্যন্ত সংখ্যাগুলোর যোগফল। ধারাবাহিকভাবে সত্য থাকা এমন বক্তব্যকে loop invariant বলা হয়।

Sentinel ও control statement

Sentinel হলো থামার বিশেষ input, যেমন `quit`। Menu-তে `while True:` ব্যবহার করলে অবশ্যই পৌঁছানো যায় এমন exit path রাখুন। Invalid input হলে কী ভুল হয়েছে বলুন।

- `break`: সবচেয়ে কাছের loop থেকে বের হয়।
- `continue`: এই iteration-এর বাকি কাজ বাদ দিয়ে পরের iteration শুরু করে।
- `pass`: কোনো কাজ করে না; placeholder। এটি `continue` নয়।

অনুশীলন

3 থেকে 1 পর্যন্ত countdown করে `Go!` দেখান। তারপর সর্বোচ্চ তিনবার চেষ্টা করা যায় এমন dummy password exercise-এর plan করুন। বাস্তব password ব্যবহার করবেন না।

8 · For loop, range ও enumerate

Item ধরে কাজ

আগেই collection থাকলে `for` দিয়ে তার item ধরে কাজ করা সহজ। String, list, dictionary, set ও range iterable।

```
scores = [65, 80, 75]
for position, score in enumerate(scores, start=1):
    print(f"Student {position}: {score}")
print("Even numbers:")
for number in range(2, 9, 2):
    print(number, end=" ")
print()
```

```
Student 1: 65
Student 2: 80
Student 3: 75
Even numbers:
2 4 6 8
```

`enumerate()` counter ও item একসঙ্গে দেয়। `start=1` শুধু দেখানোর counter বদলায়; list-এর actual index এখনো zero-based।

Range-এর stop বাদ যায়

`range(start, stop, step)`-এ stop অন্তর্ভুক্ত নয়। `range(5)` দেয় 0–4। `range(5, 0, -1)` দেয় 5–1। Step শূন্য হতে পারে না। Range সব সংখ্যা আগে থেকেই list-এ জমিয়ে রাখে না; এটি compact iterable representation।

Loop-এর else

Loop `break` ছাড়া শেষ হলে `else` চলে, এমনকি iteration শূন্য হলেও। Search-এ item পেলে break, না পেলে else-এ not found দেখানো যায়। Function-এর ভেতরে পেলে সরাসরি return করাও অনেক সময় সহজ।

চলার পথে collection বদলানো

যে list-এর ওপর iterate করছেন সেটি থেকে item সরালে কিছু item বাদ পড়তে পারে। নতুন filtered list বানান। Dictionary iterate করার সময় entry যোগ বা সরালে `RuntimeError` হতে পারে।

অনুশীলন

7-এর নামতা 1 থেকে 10 পর্যন্ত দেখান। তারপর 20-এর নিচে 3-এর গুণিতকগুলোর যোগফল বের করুন। Stop boundary কী বোঝাচ্ছে তা ব্যাখ্যা করুন।

9 · Nested loop, tracing ও pattern

ছবি মুখস্থ নয়, নিয়ম বের করুন

একটি row-তে কয়টি space, কয়টি symbol এবং কোন value লাগবে—এই সম্পর্ক বুঝলেই pattern তৈরি করা যায়। Height `n`, এক থেকে শুরু হওয়া row `r` হলে centered pyramid-এ leading space `n-r`, star `2*r-1`।

```
height = 4
for row in range(1, height + 1):
    spaces = height - row
    stars = 2 * row - 1
    print(" " * spaces + "*" * stars)
```

```
   *
  ***
 *****
*******
```

Row	শুরুর space	Star	লিখিত প্রস্থ
1	3	1	4
2	2	3	5
3	1	5	6
4	0	7	7

কল্পিত 7-width অঞ্চলে pyramid centered। শেষের অদৃশ্য trailing space ছাপানো জরুরি নয়, তাই printed line-এর দৈর্ঘ্য এক নয়।

Hollow square

```
size = 4
for row in range(size):
    for column in range(size):
        border = row in (0, size - 1) or column in (0, size - 1)
        print("*" if border else " ", end="")
    print()
```

```
****
*  *
*  *
****
```

Outer loop row বেছে নেয়। Inner loop সেই row-এর সব column ঘোরে। প্রথম/শেষ row বা প্রথম/শেষ column হলে border। Newline inner loop-এর পরে; ভেতরে দিলে প্রতিটি character আলাদা line-এ চলে যাবে।

Pattern সমাধানের recipe

1. Zero-based বা one-based coordinate বাছুন; মিশিয়ে ফেলবেন না।
2. Border ও filled region-এর শর্ত লিখুন।
3. ছোট size হাতে আঁকুন।
4. একটি row পুরো trace করুন।
5. Monospaced font-এ output মিলিয়ে দেখুন।

Right triangle-এ row `r`-এ `r` star। Inverted triangle-এ `n-r+1`। Floyd's triangle-এ একটি counter row বদলালেও চলতে থাকে। Diamond-এ width বাড়িয়ে আবার কমান; widest row ভুল করে দুবার লিখবেন না।

কাজ কত বাড়ে

Side `n`-এর square-এ প্রায় `n*n` cell দেখা হয়: $O(n^2)$। Triangle-ও quadratic growth-এর। Big-O সময়ের বৃদ্ধির ধরণ বলে, নির্দিষ্ট সেকেন্ড নয়।

অনুশীলন

Height 3-এর diamond আঁকুন। Hollow pyramid-এর বাম edge, ডান edge ও base-এর condition লিখুন। নির্বাচিত সমাধান ৩০ নম্বর অধ্যায়ে আছে।

10 · String ও text processing

String হলো sequence

String immutable: তার character সরাসরি বদলানো যায় না। Operation সাধারণত নতুন string দেয়। Index শূন্য থেকে শুরু; negative index শেষ থেকে গুনে।

```
text = "Python"
print(text[0], text[-1])
print(text[1:4])
print(text[::-1])
words = "  learn Python slowly  ".strip().split()
print(" | ".join(words))
print("banana".count("an"))
```

```
P n
yth
nohtyP
learn | Python | slowly
2
```

`text[start:stop:step]`-এ stop বাদ যায়। `[::-1]` উল্টো দিকে চলে। সীমার বাইরে একক index-এ `IndexError`; slice সাধারণত available boundary অনুযায়ী clip হয়।

Operation	কাজ
`.strip()`	দুই পাশের whitespace সরানো
`.lower()` / `.upper()`	case বদলানো
`.casefold()`	caseless তুলনায় বেশি উপযোগী normalization
`.replace(old, new)`	বদলানো নতুন text পাওয়া
`.split()`	সাধারণত whitespace ধরে ভাগ করা
`separator.join(parts)`	string-গুলো separator দিয়ে যুক্ত করা
`needle in text`	substring আছে কি না
`.startswith()` / `.endswith()`	শুরু/শেষ পরীক্ষা

`join()`-এর item string হতে হয়; number হলে আগে convert করুন। Argument ছাড়া `.split()` একাধিক whitespace একসঙ্গে ধরে। `.split(" ")` literal space ধরে; empty field থাকতে পারে।

বাংলা text ও Unicode

Python string Unicode ব্যবহার করে। একই string-এ বাংলা ও ইংরেজি রাখা যায়। কিন্তু একটি visible অক্ষর বা emoji একাধিক code point দিয়ে তৈরি হতে পারে। তাই সাধারণ slicing দিয়ে উল্টানো মানেই সব ভাষায় দৃশ্যমান অক্ষর ঠিকভাবে উল্টানো নয়।

Formatting

সাধারণভাবে `f"{name}: {score}"` সহজ। Template আলাদা থাকলে `.format()` উপকারী। একই template-এ automatic field numbering ও explicit numeric numbering মিশিয়ে ফেলবেন না।

অনুশীলন

User-এর উত্তর `.strip().casefold()` করুন। Sentence-এর `.split()` দিয়ে word count করুন এবং লিখুন যে punctuation এখনো শব্দের সঙ্গে থাকে—এটি পূর্ণ natural-language tokenizer নয়।

11 · List ও mutable object

পরিবর্তনযোগ্য ordered collection

List-এ item-এর order থাকে, duplicate থাকে এবং item বদলানো যায়। Score সংশোধন করতে এটি উপযোগী।

```
scores = [70, 85, 70]
scores.append(90)
scores[0] = 75
removed = scores.pop()
print(scores)
print(removed)
print(sorted(scores))
print(scores)
```

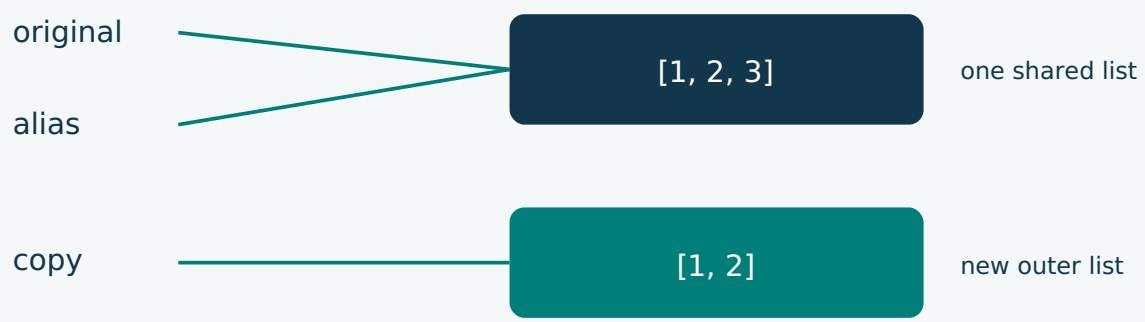
```
[75, 85, 70]
90
[70, 75, 85]
[75, 85, 70]
```

`append()` একটি object যোগ করে। `extend()` iterable-এর item-গুলো যোগ করে। `pop()` item সরিয়ে সেটি ফেরত দেয়। `remove(value)` প্রথম matching item সরায়; না পেলে `ValueError`। `.sort()` list-টিকেই বদলায় এবং `None` দেয়। `sorted()` নতুন sorted list দেয়।

Reference ও copy

```
original = [1, 2]
alias = original
copy = original.copy()
alias.append(3)
print(original)
print(copy)
```

```
[1, 2, 3]
[1, 2]
```



Assignment একই reference দেয়; shallow copy নতুন outer list বানায়।

`alias` ও `original` একই list নির্দেশ করে। একটির মাধ্যমে append করলে অন্য নাম দিয়েও পরিবর্তন দেখা যায়। `.copy()` নতুন outer list বানায়; এটি shallow copy। ভেতরে list থাকলে সেই nested object এখনো shared হতে পারে।

স্বাধীন row-এর grid বানাতে `[[0] * 3] * 3` ব্যবহার করবেন না; এতে row reference share হয়। লিখুন `[[0] * 3 for _ in range(3)]`।

Empty data ও দ্বিতীয় বৃহত্তম

Average-এর `sum(values) / len(values)`-এ list খালি হলে division by zero। `max()` ও `min()`-এরও data বা উপযুক্ত default দরকার। Empty data error হবে, special result হবে, নাকি আবার input চাইবেন—আগে ঠিক করুন।

`[9, 9, 7]`-এর দ্বিতীয় বৃহত্তম position অনুযায়ী 9, distinct value অনুযায়ী 7। Problem-এর অর্থ না লিখলে code সঠিক কি না বিচার করা যায় না। Distinct version-এ অন্তত দুটি আলাদা মান দরকার।

অনুশীলন

মূল score list অপরিবর্তিত রেখে 50 বা বেশি score-এর নতুন list বানান। `result = scores.sort()` কেন sorted list result-এ রাখে না তা ব্যাখ্যা করুন।

12 · Tuple ও unpacking

নির্দিষ্ট group

Tuple একটি immutable sequence। Coordinate বা একসঙ্গে কয়েকটি result ফেরাতে কাজে লাগে। Tuple তৈরি করে comma: এক item-এর tuple `(5,)`, শুধু `(5)` নয়।

```
point = (3, 4)
x, y = point
print(x + y)
first, *middle, last = [10, 20, 30, 40]
print(first, middle, last)
a, b = 1, 2
a, b = b, a
print(a, b)
```

```
7
10 [20, 30] 40
2 1
```

Swap-এর ডান পাশ আগে evaluate হয়, তারপর বাম নামগুলোতে binding হয়। Unpacking-এ item সংখ্যা মিলতে হয়। একটি starred target থাকলে মাঝের বাকি item-গুলো list হিসেবে নিতে পারে।

Immutable-এর সীমানা

Tuple-এর element replace করা যায় না। কিন্তু তার ভেতরে list থাকলে সেই list বদলাতে পারে। সব element hashable হলেই tuple hashable। `(1, 2)` dictionary key হতে পারে; `(1, [2])` নয়।

Fixed grouping বোঝাতে tuple নিন; সব পরিস্থিতিতে tuple দ্রুত—এমন blanket দাবি করবেন না। অনেক named field থাকলে index মনে রাখার বদলে dataclass পরিষ্কার হতে পারে।

অনুশীলন

17-কে 5 দিয়ে ভাগের quotient ও remainder একসঙ্গে ফেরত দিয়ে unpack করুন। নিজে হিসাব করার পরে `divmod()` দেখুন।

13 · Dictionary ও record

Key দিয়ে value খোঁজা

শিক্ষার্থীর নাম দিয়ে score খুঁজতে dictionary সুবিধাজনক। Key unique ও hashable হতে হয়। আধুনিক Python-এ insertion order থাকে, কিন্তু key স্বয়ংক্রিয়ভাবে sort হয় না।

```
scores = {"Amina": 82, "Rafi": 75}
scores["Nila"] = 91
scores["Rafi"] = 78
print(scores["Rafi"])
print(scores.get("Sami", "Not recorded"))
for name, score in scores.items():
    print(f"{name}: {score}")
```

```
78
Not recorded
Amina: 82
Rafi: 78
Nila: 91
```

`dictionary[key]`-এ key না থাকলে `KeyError`। `.get(key, default)` ওই missing-key পরিস্থিতি সামলায়; অন্য সব error বন্ধ করে দেয় না। একই key-তে নতুন value দিলে আগের value বদলায়।

প্রতিটি key-এর running total

```
counts = {}
for word in "red blue red green blue red".split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

```
{'red': 3, 'blue': 2, 'green': 1}
```

আগের count না থাকলে zero নিয়ে এক বাড়ানো হয়। আগের যোগফলের ধারণাই এখানে প্রতিটি word-এর জন্য আলাদা count রাখছে।

Record-এর collection

একজনের record `{"name": "Amina", "score": 82}`। অনেক শিক্ষার্থী হলে এমন dictionary-র list হতে পারে। Key consistent রাখুন এবং input boundary-তে value validate করুন। অনেক nested level জটিল হলে helper function বা class বিবেচনা করুন।

Python 3.9+ এ `left | right` merged dictionary তৈরি করে; collision হলে right-এর value জেতে। `left |= right` left-কে update করে। এটি nested dictionary-র recursive merge নয়।

অনুশীলন

কাল্পনিক নাম ও string phone number দিয়ে contact dictionary বানান। একজন যোগ করুন, একজন update করুন, missing contact নিরাপদে খুঁজুন। নমুনায় আসল ব্যক্তিগত তথ্য রাখবেন না।

14 · Set ও membership

Unique item-এর collection

Set mutable; এর element unique ও hashable হতে হয়। Position দিয়ে index করা যায় না এবং display order-এর guarantee নেই। Empty set হলো `set()`; `{}` empty dictionary।

```
python_students = {"Amina", "Rafi", "Nila"}
web_students = {"Rafi", "Sami"}
print(sorted(python_students & web_students))
print(sorted(python_students | web_students))
print(sorted(python_students - web_students))
python_students.add("Sami")
python_students.discard("Unknown")
print(len(python_students))
```

```
['Rafi']
['Amina', 'Nila', 'Rafi', 'Sami']
['Amina', 'Nila']
4
```

এখানে `sorted()` শুধু display deterministic করেছে। Set ordered হয়ে যায়নি। `|` union, `&` intersection, `-` difference এবং `^` symmetric difference—যেকোনো একটিতে আছে কিন্তু দুটোতে একসঙ্গে নয়।

`remove()`-এ absent item হলে `KeyError`; `discard()` কিছু করে না। Item না থাকা bug নাকি গ্রহণযোগ্য পরিস্থিতি—সেটা অনুযায়ী method বাছুন।

Hashable কী

Number, string ও উপযুক্ত tuple set-এর member হতে পারে। List, dictionary ও সাধারণ set পারে না। `frozenset` immutable set। Hashability হলো স্থিতিশীল hash/equality behavior-এর contract; শুধু কথায় “বদলানো যায় না” বললেই পুরো অর্থ হয় না।

Order রেখে duplicate বাদ

`list(set(items))` প্রথম আসার order রাখার প্রতিশ্রুতি দেয় না। Hashable item-এর জন্য modern Python-এ `list(dict.fromkeys(items))` order রাখে। `seen` set ও result list নিয়ে loop লিখলেও উদ্দেশ্য পরিষ্কার হয়।

অনুশীলন

দুই শিক্ষার্থীর subject set থেকে common ও আলাদা subject বের করুন। Output sort করুন, যাতে test set order-এর ওপর নির্ভর না করে।

15 · Function, argument ও return

একই নিয়ম এক জায়গায়

একটি program-এর কয়েক জায়গায় discount হিসাব করতে হলে formula বারবার লিখবেন না। একটি function-এ নিয়ম রাখুন; পরিবর্তনও এক জায়গায় হবে।

```
def discounted_price(price, percent=10):
    """Return price after a percentage discount."""
    if price < 0:
        raise ValueError("price cannot be negative")
    if not 0 <= percent <= 100:
        raise ValueError("percent must be between 0 and 100")
    return price * (1 - percent / 100)

print(f"{discounted_price(200):.2f}")
print(f"{discounted_price(200, percent=25):.2f}")
```

```
180.00
150.00
```

Definition-এর `price`, `percent` হলো parameter। Call-এর সময় দেওয়া 200 ও 25 হলো argument। Default থাকায় percent না দিলেও চলে। `percent=25` keyword argument; কোন মানের অর্থ কী তা স্পষ্ট করে।

Return আর print আলাদা

`return` caller-কে value দেয় এবং সেই function call শেষ করে। `print()` output দেখায়। শুধু print করা function-এর ফল অন্য calculation, GUI বা test-এ ব্যবহার করা কঠিন। Explicit return না থাকলে function `None` দেয়।

একটি পরিষ্কার দায়িত্ব

`calculate_total()` সাধারণত হিসাব করবে, একই সঙ্গে input নেওয়া ও file লেখা নয়। Input/output থেকে logic আলাদা রাখলে test সহজ হয়। Example-এ invalid price বা percent-এর জন্য exception দিয়ে function-এর contract প্রকাশ করা হয়েছে।

Mutable default-এর ফাঁদ

Default value definition-এর সময় একবার evaluate হয়, প্রত্যেক call-এ নয়। Fresh list চাইলে `items=[]` default দেবেন না। `items=None` দিন, প্রয়োজন হলে function-এর ভেতরে নতুন list তৈরি করুন। ৩০ নম্বর অধ্যায়ে উদাহরণ আছে।

Flexible argument

`*args` extra positional argument tuple হিসেবে ধরে। `**kwargs` extra keyword argument dictionary হিসেবে ধরে। `args` ও `kwargs` প্রচলিত নাম, keyword নয়। Flexible interface-এ কাজে লাগলেও beginner code-এ explicit parameter সাধারণত সহজ।

`return minimum, maximum` একটি tuple ফেরায়। Caller সেটি unpack করতে পারে। অনেক field হলে named dictionary বা dataclass বিবেচনা করুন।

অনুশীলন

`is_even(number)` লিখুন যা Boolean ফেরাবে; নিজে input বা print করবে না। এরপর একই function conditional ও filtering loop-এ ব্যবহার করুন।

16 · Scope, recursion ও সমস্যা ভাগ করা

নাম কোথায় পাওয়া যায়

Function-এর ভেতরে assigned নাম সাধারণত local। Python নাম খোঁজে local, enclosing function, global module এবং built-in scope-এ—সংক্ষেপে LEGB।

`global` module-level binding বোঝায়; যেকোনো বাইরের variable নয়। `nonlocal` enclosing function-এর আগে থেকে থাকা binding বোঝায়। সম্ভব হলে hidden state না বদলে parameter ও return ব্যবহার করুন।

সমস্যা: factorial

Non-negative integer `n`-এর factorial হলো `1 × 2 × ... × n`। সংজ্ঞা অনুযায়ী `0! = 1`। Recursive rule: `factorial(n) = n * factorial(n - 1)`।

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
```

Output: `24`

Call trace

`factorial(4)` অপেক্ষা করে 3-এর factorial-এর জন্য; তারপর 2, 1, 0। Base case-এ 1 ফেরে। এরপর গুণ হয়ে ফল ওঠে 1, 2, 6, 24। প্রতিটি call-এর নিজস্ব local `n` আছে।

এখানে argument integer হবে—এটি function-এর domain assumption। Negative value check আছে, কিন্তু type validation নেই। Public input boundary-তে প্রয়োজন অনুযায়ী type-ও validate করতে হয়। Type hint দিলেই runtime validation হয় না।

কখন recursion

Tree বা nested structure-এ recursion স্বাভাবিক হতে পারে। Factorial-এর জন্য loop সহজ এবং recursion-depth limit এড়ায়। শুধু beginner হিসাব চালাতে recursion limit বাড়াবেন না। Naive recursive Fibonacci একই কাজ বারবার করে; বড় input-এ iterative version বা memoization ভালো।

বড় কাজ ছোট দায়িত্বে

Expense tracker ভাগ করা যায়: amount parse → record validate → total হিসাব → file load → save → menu চালানো। এগুলো অর্থপূর্ণ দায়িত্ব। ইচ্ছামতো প্রতি দশ line পর function বানানো decomposition নয়।

অনুশীলন

Loop দিয়ে factorial লিখুন। 0, 1, 5 ও negative input দেখুন। Product শুরুতে 1 রাখার সঙ্গে recursive base case-এর সম্পর্ক ব্যাখ্যা করুন।

17 · Module, package ও virtual environment

Copy না করে reuse

Import করা যায় এমন Python file হলো module। Package module সংগঠিত করে। Standard library-তে `math`, `random`, `json`, `csv`, `pathlib`, `unittest`-এর মতো tool আছে।

```
from math import isqrt
import random

print(isqrt(81))
generator = random.Random(7)
choices = ["rock", "paper", "scissors"]
choice = generator.choice(choices)
print(choice in choices)
```

```
9
True
```

`random` game ও simulation-এর জন্য। Password বা security token-এর জন্য `secrets` ব্যবহার করুন। Seed test reproducible করতে পারে, secure করতে পারে না।

Main guard

কোনো file entry point হিসেবে চালালে তার `__name__` হয় `"__main__"`। Import করলে সাধারণত module-এর নাম হয়। তাই interactive startup `if __name__ == "__main__":` block-এ রাখুন। Helper function import করলেই যেন menu খুলে না যায়।

Project আলাদা রাখুন

Virtual environment project-এর package installation আলাদা রাখে। এই বইয়ে built-in `venv` যথেষ্ট; আলাদা `virtualenv` install বাধ্যতামূলক নয়।

```
python -m venv .venv
```

Shell	Activate করার command
Windows PowerShell	`.venv\Scripts\Activate.ps1`
Windows Command Prompt	`.venv\Scripts\activate.bat`

Shell	Activate করার command
macOS/Linux bash বা zsh	`source .venv/bin/activate`

PowerShell block করলে নির্বিচারে security setting বন্ধ করবেন না। Activate না করেও `.venv\Scripts\python.exe` দিয়ে interpreter চালানো যায়। Unix-like system-এ `.venv/bin/python`।

```
python -m pip install requests
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

Activated environment ছাড়তে `deactivate`। `pip freeze` installed distribution ও version লেখে; এটি পুরো cross-platform lockfile নয়। সব machine-এ হুবহু environment হবে এমন নিশ্চয়তা দেয় না। বইয়ের চার project standard library দিয়েই চলে; `requests` লাগবে না।

ভুল এড়ান

নিজের file `random.py` বা `json.py` রাখলে standard module shadow হতে পারে। যে interpreter দিয়ে run করছেন সেটিতেই package install করুন। `.venv` source হিসেবে share না করে recreate করুন। API key repository-তে রাখবেন না।

অনুশীলন

নতুন folder-এ venv বানান। দুটি file-এ একটি function define ও অন্যটি থেকে import করুন। Main guard দিয়ে অনাকাঙ্ক্ষিত execution বন্ধ করুন।

18 · File, path, CSV ও JSON

Program বন্ধ হলেও data থাকবে

Variable process শেষ হলে হারিয়ে যায়। File পরের run-এ data ফেরত আনতে পারে। Path-এর জন্য `pathlib.Path` এবং file বন্ধ করতে `with` ব্যবহার করুন।

```
from pathlib import Path
import json

path = Path("learning-demo.json")
record = {"name": "Amina", "topics": ["loops", "functions"]}
with path.open("w", encoding="utf-8") as file:
    json.dump(record, file, ensure_ascii=False, indent=2)
with path.open("r", encoding="utf-8") as file:
    loaded = json.load(file)
print(loaded["name"])
print(len(loaded["topics"]))
```

```
Amina
2
```

উদাহরণটি current working directory-তে `learning-demo.json` তৈরি বা replace করে। গুরুত্বপূর্ণ file-এর পাশে পরীক্ষা না করে practice folder-এ করুন।

File mode

Mode	অর্থ
`"r"`	Read; file না থাকলে error
`"w"`	Write; আগের contents মুছে যায়
`"a"`	শেষে append; না থাকলে তৈরি
`"x"`	নতুন করে create; আগে থাকলে error
`"rb"`	Binary data পড়া

Relative path current working directory থেকে resolve হয়; script-এর folder থেকেই হবে এমন নয়। Script-এর পাশের resource পেতে `Path(__file__).resolve().parent` দিয়ে শুরু করতে পারেন।

Format বাছাই

Plain text notes-এর জন্য সহজ। CSV row/column data-র জন্য। Comma দিয়ে নিজে split না করে `csv.reader` বা `csv.DictReader` নিন; quoted field-এর ভেতরে comma থাকতে পারে। CSV খুলতে `newline=""` ও encoding দিন।

JSON object, array, string, number, Boolean ও null রাখে। Python dictionary সাধারণত JSON object, list array হয়। JSON object key string। Set সরাসরি serializable নয়। Tuple save করে load করলে সাধারণত list পাওয়া যায়; প্রয়োজন হলে conversion rule বানান।

Load মানেই validate নয়

Valid JSON-ও ভুল shape-এর হতে পারে। List পেয়েছেন কি, record-এ required key আছে কি, amount সঠিক কি—দেখুন। Data পড়তে `eval()` ব্যবহার করবেন না; এটি code execute করতে পারে।

অপেক্ষাকৃত নিরাপদ save

মূল file-এ সরাসরি লেখার সময় interruption হলে আংশিক data থাকতে পারে। একই directory-তে temporary file লিখে destination replace করলে সেই ঝুঁকি কমে। তবুও backup, transaction বা multi-process coordination-এর বিকল্প নয়।

অনুশীলন

UTF-8 text file-এ তিনটি study goal লিখুন। পড়ে numbering করে দেখান। Missing-file case পরের অধ্যায়ের exception handling দিয়ে সামলান।

19 · Exception, validation ও debugging

অসম্পূর্ণ input স্বাভাবিক

Validation দেখে data নিয়ম মানছে কি না। Exception handling ব্যর্থ operation থেকে recovery করে। একটির বদলে আরেকটি নয়; একসঙ্গে দরকার হয়।

```
def parse_positive_integer(text):
    try:
        value = int(text)
    except ValueError as error:
        raise ValueError("enter a whole number") from error
    if value <= 0:
        raise ValueError("number must be positive")
    return value

for text in ["12", "0", "twelve"]:
    try:
        print(parse_positive_integer(text))
    except ValueError as error:
        print(f"Invalid: {error}")
```

```
12
Invalid: number must be positive
Invalid: enter a whole number
```

Try block-এর অর্থ

`try`-এ ব্যর্থ হতে পারে এমন operation। `except` নির্দিষ্ট exception সামলায়। Try স্বাভাবিকভাবে সম্পন্ন হলে `else` চলে। `finally` সাধারণ Python execution-এ try ছাড়ার সময় চলে, সাধারণ error ও return path-সহ। Power failure, abrupt process termination বা `os._exit()`-এর ক্ষেত্রে সাধারণ এই guarantee প্রযোজ্য নয়।

যে error বোঝেন সেটি কাছে catch করুন। Bare `except:` `KeyboardInterrupt`-ও ধরে ফেলতে পারে, ফলে program বন্ধ করাও কঠিন হয়। `except Exception:` দিয়ে সব bug লুকিয়ে ফেলাও ভালো নয়।

Traceback পড়া

নিচের শেষ line exception-এর type ও message বলে। উপরের frame-গুলো দেখায় কোন call থেকে কোথায় পৌঁছেছে। নিজের সংশ্লিষ্ট line খুঁজুন, actual value দেখুন, ছোট failing case বানান।

Error/লক্ষণ	প্রশ্ন
`NameError`	এই scope-এ নামটি define হয়েছে?
`TypeError`	value-গুলোর ওপর operation চলে?

Error/লক্ষণ	প্রশ্ন
`IndexError`	index sequence-এর মধ্যে আছে?
`KeyError`	key আছে?
`ValueError`	operation এই value গ্রহণ করে?
Error নেই, ফল ভুল	algorithm বা boundary ভুল?

Debugging-এর ধাপ

1. সঠিক ফল কী হওয়া উচিত লিখুন।
2. সবচেয়ে ছোট failing example বানান।
3. Temporary print বা `breakpoint()` দিয়ে value দেখুন।
4. একটি iteration বা call trace করুন।
5. লক্ষণ নয়, মূল কারণ ঠিক করুন।
6. একই bug ফেরত আসা ঠেকাতে test যোগ করুন।

সাধারণ arithmetic-এ zero দিয়ে division undefined। শুধু কোনো exercise “infinite” দেখাতে বলেছে বলে সেটিকে বৈধ mathematical result হিসেবে শেখাবেন না। Operation সম্ভব নয় বলে জানান।

অনুশীলন

`FileNotFoundError` catch করে কোন path পাওয়া যায়নি তা জানান। সব exception ধরে চুপচাপ empty data ফেরাবেন না; এতে নষ্ট file-ও নতুন empty file দিয়ে overwrite হতে পারে।

20 · Test ও program design

শুধু স্বাভাবিক input নয়

প্রতিটি test-এ input, expected result ও কারণ থাকে। Normal value, boundary, empty data, duplicate ও invalid value ভাবুন। Pure function keyboard, network বা hidden state-এর ওপর নির্ভর করে না বলে test সহজ।

```
import unittest


def second_largest_distinct(values):
    unique = sorted(set(values))
    if len(unique) < 2:
        raise ValueError("need at least two distinct values")
    return unique[-2]


class RankingTests(unittest.TestCase):
    def test_normal(self):
        self.assertEqual(second_largest_distinct([4, 9, 7]), 7)

    def test_duplicates(self):
        self.assertEqual(second_largest_distinct([9, 9, 7]), 7)

    def test_insufficient(self):
        with self.assertRaises(ValueError):
            second_largest_distinct([5, 5])


if __name__ == "__main__":
    unittest.main()
```

এই file চালালে তিনটি test ও সফল হলে `OK` পাওয়া যায়। `unittest` সাধারণত progress standard error stream-এ লেখে; terminal-এ সেটিও দেখা যায়।

কী প্রমাণ হলো

Test-গুলো দ্বিতীয় বৃহত্তম **distinct** value-এর interpretation পরীক্ষা করে। Mixed incomparable type, unhashable object বা সব possible dataset পরীক্ষা করেনি। Function-এর contract-এ domain লিখুন: এখানে comparable, hashable numeric value-এর finite collection।

Assert বনাম validation

Development-এ `assert` উপকারী, কিন্তু optimized mode-এ এটি বাদ যেতে পারে। Untrusted input reject করার বাধ্যতামূলক নিয়ম শুধু assert-এর ওপর রাখবেন না; explicit check ও exception দিন।

বদলানো সহজ রাখুন

Parsing, calculation ও presentation আলাদা করুন। বড় এক loop-এ সব feature না ঢুকিয়ে অর্থপূর্ণ helper বানান। সমস্যা না থাকতেই জটিল class hierarchy বানানোর প্রয়োজন নেই।

Complexity-এর ধারণা

একবার traversal সাধারণত O(n)। Sorting প্রায় O(n log n)। Double scan $O(n^2)$ হতে পারে। Dictionary/set membership সাধারণ hashing assumption-এ average O(1), শর্তহীন guarantee নয়। আগে সঠিক ফল, পরে measurement দেখে optimization।

অনুশীলন

Average function-এর test লিখুন। Empty list-এর আচরণ আগে ঠিক করুন এবং implementation-এর আগে সেই test লিখুন।

21 · Class ও object-oriented চিন্তা

State আর কাজ একসঙ্গে

Class এক ধরনের object-এর সংজ্ঞা। Instance সেই ধরনের নির্দিষ্ট object। State এবং তা রক্ষা বা পরিবর্তনের কাজ একসঙ্গে থাকলে class সুবিধাজনক; শুধু noun দেখলেই class বানাতে হবে না।

```
class StudySession:
    category = "learning"

    def __init__(self, topic):
        self.topic = topic
        self.minutes = 0

    def add_minutes(self, amount):
        if amount <= 0:
            raise ValueError("minutes must be positive")
        self.minutes += amount

    def summary(self):
        return f"{self.topic}: {self.minutes} minutes"


session = StudySession("Python")
session.add_minutes(25)
session.add_minutes(10)
print(session.summary())
```

```
Python: 35 minutes
```

Method call অনুসরণ

`session.add_minutes(25)` instance-টিকে প্রথম argument হিসেবে দেয়; প্রচলিত নাম `self`। এটি keyword নয়, কিন্তু নাম বদলালে পাঠকের অসুবিধা হয়।

`__init__` object তৈরি হওয়ার পরে initialize করে। একে কথায় constructor বলা হলেও object creation-এর বিশেষ কাজ `__new__`-এর। `__init__` নতুন object return করবে না; এর return value `None` হতে হয়।

Shared ও নিজস্ব attribute

`category` class attribute। `topic` ও `minutes` instance-এ রাখা। `session.category = "practice"` সাধারণত ওই instance-এ attribute বানিয়ে class attribute shadow করে; class-এর value সবার জন্য বদলায় না।

প্রতি instance-এর আলাদা list চাইলে class-level list নয়, `__init__`-এ নতুন list দিন। নইলে একজনের পরিবর্তন অন্যজনেও দেখা যেতে পারে।

তিন method

- Instance method `self` পায়; instance state নিয়ে কাজ করে।
- `@classmethod` `cls` পায়; alternative constructor বানাতে পারে।
- `@staticmethod` নিজে থেকে `self` বা `cls` পায় না; class-সংশ্লিষ্ট utility হতে পারে।

অনুশীলন

Title ও author-সহ `Book` বানান। Description ফেরানো method দিন। দুটি instance বানিয়ে একটির title বদলান; অন্যটি বদলাচ্ছে কি না দেখুন।

22 · Inheritance, property ও special method

সম্পর্কটি সত্যিই “এক ধরনের” কি

Child class parent-এর behavior বাড়াতে পারে। কিন্তু শুধু code reuse-এর জন্য inheritance নেওয়া সবসময় ভালো নয়। Child কি parent-এর জায়গায় যুক্তিসংগতভাবে ব্যবহার করা যায়? না গেলে composition ভাবুন: এক object আরেকটি ধারণ করে ও তার কাজ ব্যবহার করে।

```
class Resource:
    def __init__(self, title):
        self.title = title

    def describe(self):
        return self.title


class Video(Resource):
    def __init__(self, title, minutes):
        super().__init__(title)
        self.minutes = minutes

    def describe(self):
        return f"{self.title} ({self.minutes} min)"


lesson = Video("Loops", 12)
print(lesson.describe())
print(isinstance(lesson, Resource))
```

```
Loops (12 min)
True
```

`super()` method resolution order বা MRO অনুসরণ করে। Single inheritance-এ parent-এর method-এ পৌঁছানো সহজে বোঝা যায়। Multiple inheritance-এ এটি কোনো স্থির parent-এর সংক্ষিপ্ত নাম নয়; cooperative method ও compatible argument দরকার।

Single inheritance-এ একটি direct base, multiple-এ একাধিক, multilevel-এ chain। শুরুতে ছোট ও shallow design রাখুন।

Property দিয়ে নিয়ম রক্ষা

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = value

    def __str__(self):
        return f"{self.celsius:.1f} C"


print(Temperature(25))
```

```
25.0 C
```

Setter value সংরক্ষণের আগে absolute zero-এর নিচে কি না দেখে। `_celsius` internal detail বোঝানোর convention, security barrier নয়। প্রতিটি attribute-এর জন্য যান্ত্রিকভাবে getter/setter দরকার নেই; অনেক ক্ষেত্রে plain public attribute যথেষ্ট।

Special method-এর contract

`__str__` readable text দেয়; `__repr__` developer-friendly representation। `__len__` non-negative integer দিতে হয়। `__add__` `+`-এর behavior দিতে পারে। Unsupported operand হলে `NotImplemented` ফেরান, যাতে Python উপযুক্ত reflected operation চেষ্টা করতে পারে। Operator-কে অপ্রত্যাশিত অর্থ দেবেন না।

`@dataclass` simple record-এর initialization ও representation-এর boilerplate কমায়। এটি নিজে type validation করে না বা সব nested data immutable বানায় না।

অনুশীলন

x ও y coordinate-সহ vector design করুন। Vector যোগ ও string representation কী হবে লিখুন। String যোগ করতে চাইলে কী হবে সেটিও নির্ধারণ করুন।

23 · Comprehension, iterator ও functional tool

তিন ধরনের সাধারণ কাজ

প্রতিটি item বদলানো হলো transform, শর্ত মানা item রাখা হলো select, সব item থেকে একটি result বানানো হলো accumulate। কাজটি সবচেয়ে পরিষ্কারভাবে প্রকাশ করুন।

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
squares = [number * number for number in numbers]
even = [number for number in numbers if number % 2 == 0]
print(squares)
print(even)
print(list(map(lambda number: number * 2, numbers)))
print(list(filter(lambda number: number > 3, numbers)))
print(reduce(lambda total, number: total + number, numbers, 0))
print(sum(number * number for number in numbers))
```

```
[1, 4, 9, 16, 25]
[2, 4]
[2, 4, 6, 8, 10]
[4, 5]
15
55
```

Comprehension পড়ার নিয়ম

`[expression for item in iterable if condition]`: প্রতি item নিন, condition সত্য হলে expression-এর ফল রাখুন। Variable-এর নাম মিলতে হবে। `[i for item in values]` সাধারণত ভুল; বাইরে আলাদা `i` থাকলেও সম্ভবত কাঙ্ক্ষিত কাজ নয়।

Set ও dictionary comprehension-ও আছে। অনেক nested loop ও condition এক line-এ গুঁজবেন না। Normal loop ব্যবহার করা দুর্বলতা নয়।

Iterator ও generator

Python 3-এ `map()` ও `filter()` iterator দেয়, list নয়। Iterate করলে consume হয়। Generator expression lazily item তৈরি করে। Generator function-এ `yield` execution pause করে; পরে সেখান থেকেই চলতে পারে।

Laziness memory বাঁচাতে পারে, কিন্তু error-ও iteration পর্যন্ত পিছিয়ে দিতে পারে। এক result বারবার ঘুরতে হলে list বানান বা নতুন iterator তৈরি করুন।

Lambda, closure, decorator

Lambda single-expression function। একাধিক statement বা documentation দরকার হলে `def` পরিষ্কার। Closure enclosing function-এর variable ব্যবহার ধরে রাখে। Decorator callable নেয় এবং replacement callable ফেরায়; logging বা timing যোগ করতে পারে। প্রথম useful program লেখার আগে decorator আয়ত্ত করা বাধ্যতামূলক নয়।

`reduce()` running result-এর সঙ্গে পরের item combine করে। সাধারণ যোগে `sum()`, সর্বোচ্চে `max()` বেশি পরিষ্কার। কঠিন দেখাতে সহজ কাজ জটিল করবেন না।

অনুশীলন

Name list থেকে strip করা non-empty নাম রাখুন। আগে loop, পরে comprehension। Strip-এর আগে শুধু truthiness পরীক্ষা করলে whitespace-only নাম কেন থেকে যায় তা বলুন।

24 · Type hint, match ও walrus

Intent প্রকাশ

Type annotation পাঠক ও static analysis tool-কে expected type বোঝায়। Python নিজে সব annotation runtime-এ enforce করে না।

```
def describe_status(code: int) -> str:
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not found"
        case 500:
            return "Server error"
        case _:
            return "Unknown status"


scores: list[int] = [72, 81, 90]
identifier: int | str = "ID-101"
print(describe_status(404))
if (count := len(scores)) >= 3:
    print(f"Recorded {count} scores")
```

```
Not found
Recorded 3 scores
```

`list[int]` Python 3.9+। `int | str` ও `match` 3.10+। পুরোনো reference-এ `typing.List`, `typing.Union` দেখবেন; চিনে রাখুন, নতুন baseline-এ modern syntax ব্যবহার করতে পারেন।

Match শুধু switch নয়

Literal case switch-এর মতো মনে হলেও এটি structural pattern matching: sequence, mapping ও object structure-ও match করতে পারে। Pattern-এ bare name সাধারণত আগের variable-এর সঙ্গে compare না করে capture করে। Literal বা qualified constant ব্যবহার করে উদ্দেশ্য স্পষ্ট করুন। Case-এর পরে fall-through বন্ধ করতে `break` লাগে না।

Walrus operator

Python 3.8 থেকে `:=` expression-এর মধ্যে assignment করে। এখানে count একবার হিসাব করে condition ও message-এ ব্যবহার হচ্ছে। শুধু line ছোট করতে ব্যবহার করবেন না; আলাদা assignment অনেক সময় ভালো।

Hint validation নয়

`amount: int` লিখলেই `"ten"` reject হয় না। Input boundary-তে check দিন। Static checker অনেক ভুল ধরলেও algorithm সঠিক প্রমাণ করে না।

অনুশীলন

`list[float]` নিয়ে `float` ফেরানো average function annotate করুন। Empty list-এর আচরণ কী, কোনটি runtime code দেখে এবং কোনটি শুধু hint—আলাদা করে লিখুন।

25 · প্রজেক্ট: নির্ভরযোগ্য calculator

লক্ষ্য ও contract

দুটি finite number এবং অনুমোদিত operator নিয়ে হিসাব করব। ভুল input-এ ব্যাখ্যা দেব, আবার চেষ্টা করতে দেব, এবং quit করলে থামব। Input-কে code হিসেবে চালাতে `eval()` নয়।

Flow: প্রথম value/quit → operator → দ্বিতীয় value → validate → calculation → ফল বা error → আবার prompt।

```
import math


def calculate(left, operator, right):
    if not math.isfinite(left) or not math.isfinite(right):
        raise ValueError("numbers must be finite")
    if operator == "+":
        result = left + right
    elif operator == "-":
        result = left - right
    elif operator == "*":
        result = left * right
    elif operator == "/":
        if right == 0:
            raise ValueError("division by zero is undefined")
        result = left / right
    else:
        raise ValueError("choose +, -, * or /")
    if not math.isfinite(result):
        raise ValueError("result is too large")
    return result


def main():
    print("Calculator — type q as the first value to quit.")
    while True:
        first = input("First value: ").strip()
        if first.casefold() == "q":
            print("Goodbye.")
            break
        try:
            left = float(first)
            operator = input("Operation (+ - * /): ").strip()
            right = float(input("Second value: "))
            result = calculate(left, operator, right)
        except ValueError as error:
            print(f"Cannot calculate: {error}")
        else:
            print(f"Result: {result:g}")
```

```
if __name__ == "__main__":
    main()
```

চালিয়ে দেখুন

প্রথমে `12`, তারপর `/`, তারপর `4` দিলে `Result: 3`। একইভাবে divisor `0` দিলে division undefined-এর message। প্রথম value হিসেবে `q` দিলে program শেষ।

Design বোঝা

`calculate()`-এ numerical rule, `main()`-এ কথোপকথন। ফলে keyboard ছাড়াই calculation test করা যায়। Expected parsing/domain error-এর জন্য `ValueError` ধরা হয়েছে; unrelated bug লুকানো হয়নি। `float()` `nan` বা infinity parse করতে পারে বলে `math.isfinite()` দিয়ে reject করা হয়েছে।

এই calculator float ব্যবহার করে; financial accounting-এর জন্য নয়। Display round করা মানে arithmetic exact নয়।

Test	প্রত্যাশা
`12 / 4`	3
`2 - 5`	-3
Zero দিয়ে ভাগ	পরিষ্কার error
Unknown operator	অনুমোদিত option জানাবে
অক্ষর input	error, পরে আবার চেষ্টা
NaN বা infinity	reject
Quit	loop শেষ

বাড়তি কাজ

Exponentiation যোগ করার আগে negative base, fractional exponent ও overflow-এর policy ঠিক করুন। History চাইলে calculation function-এ গোপনে file write না ঢুকিয়ে input/output layer-এ record তৈরি করুন।

26 · প্রজেক্ট: quiz ও score report

প্রশ্নকে data হিসেবে রাখুন

প্রতিটি প্রশ্নের জন্য আলাদা branch নয়। Question data-র ওপর একটি reusable loop চালাব। Answer normalize করব, correct গুনব এবং ভুলগুলো review-তে দেখাব।

```
QUESTIONS = [
    {"prompt": "Type of input() result? ", "answer": "str"},
    {"prompt": "Result of 7 // 2? ", "answer": "3"},
    {"prompt": "Keyword that returns a function result? ", "answer": "return"},
]


def normalize(text):
    return text.strip().casefold()


def run_quiz(questions):
    correct = 0
    missed = []
    for number, question in enumerate(questions, start=1):
        response = input(f"{number}. {question['prompt']}")
        if normalize(response) == normalize(question["answer"]):
            correct += 1
            print("Correct.")
        else:
            print("Not this time.")
            missed.append(question)
    return correct, missed


def main():
    if not QUESTIONS:
        print("No questions available.")
        return
    correct, missed = run_quiz(QUESTIONS)
    percentage = correct / len(QUESTIONS) * 100
    print(f"Score: {correct}/{len(QUESTIONS)} ({percentage:.1f}%)")
    for question in missed:
        print(f"Review: {question['prompt']}Answer: {question['answer']}")


if __name__ == "__main__":
    main()
```

একটি attempt trace

Input `str`, `3`, `print` হলে প্রথম দুটি ঠিক। Strip করার পর `str` মেলে। শেষটি ভুল: result দেখানো আর return করা আলাদা। Score `2/3 (66.7%)`; percentage display-এর জন্য round হয়েছে।

Logic ও data আলাদা কেন

প্রশ্নগুলো data; loop algorithm। ভবিষ্যতে JSON question bank load করলেও score logic বদলাতে হবে না। Empty question check না থাকলে percentage-এ zero দিয়ে division হতে পারে।

Normalization একটি policy। Keyword answer-এর ক্ষেত্রে casefold ভালো, কিন্তু case-sensitive identifier বা exact-output প্রশ্নে অতি normalization ভুল পার্থক্য মুছে দিতে পারে।

Test ও উন্নয়ন

সব সঠিক, সব ভুল, mixed, extra space, capitalization এবং empty question list পরীক্ষা করুন। সত্যিই সমার্থক হলে accepted alternative রাখুন। Shuffle যোগ করলে প্রশ্ন ও answer যেন একসঙ্গে থাকে।

27 · প্রজেক্ট: JSON-এ খরচের হিসাব

লক্ষ্য

লোকাল command-line tool দিয়ে expense যোগ, list, category summary ও save/reload করব। 100 পয়সা = 1 টাকা ধরে integer minor unit রাখব। এটি educational single-user tool, banking application নয়।

Data model আগে

Record-এ ISO date text, non-empty category ও positive integer `amount_minor` থাকবে। Amount parsing, saved data validation, totals, load, save ও menu আলাদা function। Temporary file-এ লিখে destination replace করা হবে।

Flow: data load → menu → add/list/summary/quit → input check → add হলে save → আবার menu।

```
from datetime import date
from decimal import Decimal, InvalidOperation, localcontext
import json
from pathlib import Path

DATA_PATH = Path(__file__).resolve().with_name("expenses.json")


def parse_amount(text):
    if len(text) > 64:
        raise ValueError("amount text is too long")
    try:
        amount = Decimal(text)
    except InvalidOperation as error:
        raise ValueError("enter a decimal amount") from error
    if not amount.is_finite() or amount <= 0:
        raise ValueError("amount must be positive and finite")
    if amount > Decimal("1000000000"):
        raise ValueError("amount exceeds this demo's limit")
    if amount < Decimal("0.01"):
        raise ValueError("minimum amount is one paisa")
    with localcontext() as context:
        context.prec = max(28, len(amount.as_tuple().digits) + 3)
        minor = amount * 100
        if minor != minor.to_integral_value():
            raise ValueError("use at most two decimal places")
        return int(minor)


def money(minor):
    return f"Tk {minor // 100}.{minor % 100:02d}"
```

```
def validate_records(records):
    if not isinstance(records, list):
        raise ValueError("saved data must be a list")
    for record in records:
        if not isinstance(record, dict):
            raise ValueError("each record must be an object")
        if set(record) != {"date", "category", "amount_minor"}:
            raise ValueError("record fields do not match")
        if not isinstance(record["date"], str):
            raise ValueError("date must be text")
        try:
            parsed = date.fromisoformat(record["date"])
        except ValueError as error:
            raise ValueError("invalid saved date") from error
        if parsed.isoformat() != record["date"]:
            raise ValueError("use YYYY-MM-DD dates")
        category = record["category"]
        if not isinstance(category, str) or not category.strip():
            raise ValueError("category must be non-empty text")
        amount = record["amount_minor"]
        if type(amount) is not int or not 0 < amount <= 100000000000:
            raise ValueError("invalid minor-unit amount")
    return records


def load_records(path):
    try:
        with path.open("r", encoding="utf-8") as file:
            records = json.load(file)
    except FileNotFoundError:
        return []
    return validate_records(records)


def save_records(path, records):
    validate_records(records)
    temporary = path.with_suffix(path.suffix + ".tmp")
    with temporary.open("w", encoding="utf-8") as file:
        json.dump(records, file, ensure_ascii=False, indent=2)
        file.write("\n")
    temporary.replace(path)


def totals_by_category(records):
    totals = {}
    for record in records:
        category = record["category"]
        totals[category] = totals.get(category, 0) + record["amount_minor"]
    return totals


def main():
    try:
        records = load_records(DATA_PATH)
    except (OSError, ValueError) as error:
        print(f"Cannot load data: {error}")
        print("Repair or back up the file first. Nothing was overwritten.")
        return
    while True:
```

```
        print("\n1 Add | 2 List | 3 Summary | 4 Quit")
        choice = input("Choice: ").strip()
        if choice == "1":
            category = input("Category: ").strip().casefold()
            try:
                if not category:
                    raise ValueError("category cannot be blank")
                amount = parse_amount(input("Amount in taka: ").strip())
                new_record = {
                    "date": date.today().isoformat(),
                    "category": category,
                    "amount_minor": amount,
                }
                updated = records + [new_record]
                save_records(DATA_PATH, updated)
            except (OSError, ValueError) as error:
                print(f"Not saved: {error}")
            else:
                records = updated
                print("Saved.")
        elif choice == "2":
            if not records:
                print("No expenses yet.")
            for record in records:
                print(record["date"], record["category"],
                      money(record["amount_minor"]))
        elif choice == "3":
            totals = totals_by_category(records)
            for category, amount in sorted(totals.items()):
                print(f"{category}: {money(amount)}")
            print(f"Total: {money(sum(totals.values()))}")
        elif choice == "4":
            print("Goodbye.")
            break
        else:
            print("Choose 1, 2, 3 or 4.")


if __name__ == "__main__":
    main()
```

গুরুত্বপূর্ণ সিদ্ধান্তগুলো

Exact amount: আগে float না বানিয়ে `Decimal` দিয়ে text parse। `12.50` হয় 1250 পয়সা। `0.001` silently round নয়, reject। Input length ও এক record-এ সর্বোচ্চ এক বিলিয়ন টাকা সীমা আছে। Local decimal context প্রয়োজনীয় significant digit ধরে conversion করে; লম্বা fractional value ভুল করে valid amount-এ round হয় না।

Data হারাব না: malformed JSON বা invalid schema পেলে empty list ধরে এগোই না। Message দিয়ে program থামে। নইলে পরের save-এ আগের অজানা data মুছে যেতে পারত।

আগে save, পরে active state: `updated` candidate list। Save সফল হলেই `records = updated`। Save ব্যর্থ হলে program-এর memory-তে expense জমা হয়েছে এমন ভান করা হয় না।

Date: নতুন entry computer-এর local date নেয়। Loader canonical `YYYY-MM-DD` চায়। Timezone-aware accounting বা backdated entry এই সংস্করণে নেই।

Boolean ফাঁদ: `bool` হলো `int`-এর subclass। তাই `isinstance(True, int)` সত্য। JSON Boolean যেন amount না হয়, সেজন্য validator-এ `type(amount) is int` ব্যবহার করা হয়েছে।

উদাহরণ session

`food` category-তে `120.50`, `travel`-এ `30` যোগ করুন। Summary হবে `food: Tk 120.50`, `travel: Tk 30.00`, মোট `Tk 150.50`। Quit করে আবার চালিয়ে data আছে কি না দেখুন। `expenses.json` script-এর পাশেই থাকবে।

সীমাবদ্ধতা

এক data file-এ একসঙ্গে একটিই instance চালান। Fixed temporary filename ও replace approach multi-user locking বা full crash durability দেয় না। Backup রাখুন। Authentication, encryption, edit/delete ও database transaction নেই। বড় ব্যবহারে SQLite, migration, backup ও privacy requirement ভাবতে হবে।

Test plan

Empty storage, valid save/reload, দুই category, fractional-paisa, negative, infinity, broken JSON, wrong record field ও unwritable path পরীক্ষা করুন। Test-এর জন্য temporary directory ব্যবহার করুন, আসল expense file নয়।

28 · প্রজেক্ট: assistant ও reply draft

Human control বজায় রাখুন

Voice assistant বা auto-reply chatbot-এ microphone, recognition, text-to-speech, network, credential, chat parsing ও UI automation—অনেক স্বাধীন ব্যবস্থা থাকে। আগে text core ঠিক করুন; পরে একটি করে adapter যোগ করুন।

এই project offline rule-based **reply draft** বানায়, নিজে message পাঠায় না। এটি complete speech-recognition বা language-model system নয়। Standard library ছাড়া কিছু লাগে না।

```
from datetime import date


def draft_reply(message):
    normalized = message.strip().casefold()
    if not normalized:
        raise ValueError("message cannot be empty")
    if "thank" in normalized:
        return "You're welcome! Happy to help."
    if "meeting" in normalized:
        return "Thanks for the update. Please confirm the time and agenda."
    if normalized in {"hi", "hello", "hey"}:
        return "Hello! How can I help?"
    return "Thanks for your message. I'll review it and reply soon."


def respond(command):
    normalized = command.strip().casefold()
    if normalized == "help":
        return "Commands: date, draft <message>, help, quit"
    if normalized == "date":
        return date.today().isoformat()
    if normalized.startswith("draft "):
        message = command.strip()[6:]
        return "DRAFT ONLY: " + draft_reply(message)
    return "Unknown command. Type help."


def main():
    print("Local assistant. No message is sent automatically.")
    while True:
        command = input("> ")
        if command.strip().casefold() == "quit":
            print("Goodbye.")
            break
        try:
            print(respond(command))
        except ValueError as error:
            print(f"Cannot draft: {error}")
```

```
if __name__ == "__main__":
    main()
```

উদাহরণ

`draft Thanks for your help` দিলে `DRAFT ONLY: You're welcome! Happy to help.` পাওয়া যাবে। User দেখে সিদ্ধান্ত নেবেন। `date` run-এর সময়কার local date দেয়; বইয়ে hard-code করা হয়নি।

কী শিখলাম

Core function input থেকে result বানায়; interactive loop আলাদা। Rule-based matching-এর সীমা আছে। Text-এ `thank` আছে কি না দেখা semantic understanding নয়। প্রথম matching rule-ই জেতে। এসব দৃশ্যমান design choice, গোপন বুদ্ধিমত্তা নয়।

নিরাপদ extension architecture

Input adapter → command router → অনুমোদিত tool/draft generator → preview → explicit confirmation → action। Model-এর suggestion untrusted input; সেটি shell command চালানো বা message পাঠানোর অনুমতি নয়। Tool execution text generation থেকে আলাদা রাখুন।

Microphone permission নিন এবং stop control দিন। Recording device ছাড়ে কি না পরিষ্কার বলুন। External AI-তে chat history পাঠানোর আগে consent নিন, প্রয়োজনের কম text পাঠান, key environment variable-এ রাখুন। Timeout ও rate limit সামলান। Key বা private conversation repository-তে দেবেন না।

Mouse coordinate-ভিত্তিক automation-এ layout বদলালে ভুল recipient-কে text পাঠানো হতে পারে। Supported API, platform rule ও explicit confirmation পছন্দ করুন। Deduplication ও loop protection দিন, যাতে দুটি automated account অনন্ত reply না করে।

অনুশীলন

`draft_reply()` না বদলে `time` command যোগ করুন। Blank message reject হয় এবং draft review করা যায়—এর test লিখুন। পরে draft generation বদলালেও confirmation control বজায় রাখুন।

29 · একশটি practice challenge

প্রতিটি সমস্যার input domain, স্বাভাবিক case, boundary ও প্রযোজ্য হলে invalid case লিখুন। সবগুলোর পূর্ণ সমাধান এই বইয়ে নেই। পরের অধ্যায়ে নির্বাচিত worked solution আছে; আগের lesson ও project-ও সংশ্লিষ্ট ধারণা দেখিয়েছে। একসঙ্গে সব শেষ না করে ধাপে ধাপে করুন।

ভিত্তি · 01–10

1. তিন line-এ নিজের পরিচয় ও শেখার লক্ষ্য দেখান।
2. Non-negative dimension-এর আয়তক্ষেত্রের area ও perimeter বের করুন।
3. Non-negative seconds থেকে hour, minute ও second বের করুন।
4. Temporary name ছাড়া দুটি variable swap করে evaluation order বোঝান।
5. Celsius থেকে Fahrenheit ও উল্টো conversion করুন।
6. Validated discount percentage দিয়ে final price বের করুন।
7. Simple interest বের করে rate-এর সঙ্গে time unit মিলিয়ে লিখুন।
8. Whole-number string integer-এ convert করুন; invalid text সামলান।
9. Measurement তিন decimal place-এ দেখান; exact arithmetic দাবি করবেন না।
10. Phone number text রাখার কারণ উদাহরণে দেখান।

সিদ্ধান্ত · 11–20

1. Integer even না odd বের করুন।
2. Number positive, zero না negative তা বলুন।
3. Sort ছাড়া তিন সংখ্যার বৃহত্তম বের করুন।
4. Valid mark থেকে descending threshold দিয়ে grade দিন।
5. Gregorian leap-year rule প্রয়োগ করুন।
6. তিন positive side দিয়ে triangle সম্ভব কি না দেখুন।
7. Unknown choice reject করা menu বানান।
8. Chained comparison দিয়ে age range check করুন।
9. Yes/no normalize করুন; অন্য input reject করুন।
10. স্পষ্ট English-letter policy দিয়ে character vowel কি না দেখুন।

Loop ও number · 21–35

1. Non-negative `n`-এর জন্য 1 থেকে `n` যোগ করুন।

2. Iterative factorial লিখুন; negative input reject করুন।
3. নির্ধারিত inclusive multiplier পর্যন্ত নামতা দেখান।
4. Prime check করুন; 2-এর ছোট সংখ্যা prime নয়।
5. Inclusive interval-এর prime-গুলো দেখান।
6. শুরুর convention লিখে প্রথম `n` Fibonacci term দিন।
7. Euclid algorithm দিয়ে GCD করুন।
8. LCM করুন; কোনো input zero হলে আচরণ ঠিক করুন।
9. Sign বজায় রেখে integer-এর digit reverse করুন।
10. Zero ও negative sign ঠিকভাবে সামলে digit count করুন।
11. Absolute value-এর decimal digit-গুলোর যোগফল করুন।
12. Non-negative integer Armstrong number কি না দেখুন।
13. Positive integer perfect number কি না দেখুন।
14. Sentinel না আসা পর্যন্ত value যোগ করুন; sentinel যোগ করবেন না।
15. Non-negative integer exponent-এর জন্য power হিসাব করুন।

String · 36–45

1. Whitespace-separated word গুনুন।
2. Normalization policy লিখে phrase palindrome পরীক্ষা করুন।
3. English alphabetic letter-এর vowel ও consonant গুনুন।
4. Character-frequency dictionary বানান।
5. সর্বাধিক আসা character বের করুন; tie-তে প্রথম appearance নিন।
6. প্রথম appearance-এর order রেখে repeated character বাদ দিন।
7. দুটি normalized phrase anagram কি না দেখুন।
8. Target word বদলান, বড় word-এর অংশ যেন ভুল করে না বদলায়।
9. Longest word বের করুন; tie ও punctuation-এর policy লিখুন।
10. Non-empty username-এর সহজ documented rule validate করুন।

Collection · 46–60

1. Empty input reject করে average বের করুন।
2. দ্বিতীয় বৃহত্তম distinct value বের করুন।
3. Input order রেখে duplicate বাদ দিন।
4. List chunk করুন; non-positive size reject করুন।
5. `k` position list rotate করুন; direction ও empty case ঠিক করুন।

6. Nested list-এর এক level flatten করুন।
7. দুটি sorted numeric list merge করে sorted output দিন।
8. Independent row-সহ 3×3 grid বানান।
9. Minimum ও maximum unpack করা যায় এমন pair-এ ফেরান।
10. Dictionary দিয়ে word গুনুন।
11. Dictionary invert করুন; একই value-এর সব key সংরক্ষণ করুন।
12. দুটি dictionary merge করে collision policy লিখুন।
13. Set intersection ও difference দিয়ে group তুলনা করুন।
14. দুটি set-এর ঠিক একটিতে থাকা value বের করুন।
15. Student record score অনুযায়ী, তারপর name অনুযায়ী sort করুন।

Function ও design · 61–70

1. Reusable positive-integer parser লিখুন।
2. Print না করে Boolean ফেরানো function লিখুন।
3. Quotient ও remainder ফেরান; zero divisor reject করুন।
4. Mutable default-এর নিরাপদ বিকল্প দেখান।
5. Positional, keyword ও default argument একই ছোট interface-এ দেখান।
6. `*args` দিয়ে total করুন, built-in `sum` shadow করবেন না।
7. Multiplier মনে রাখে এমন closure লিখুন।
8. `functools.wraps` দিয়ে metadata রাখা decorator লিখুন।
9. Iterative ও recursive factorial-এর practical limit তুলনা করুন।
10. Text analysis-কে parsing, counting ও reporting-এ ভাগ করুন।

Pattern · 71–80

1. Right-aligned star triangle লিখুন।
2. Inverted left-aligned triangle লিখুন।
3. Centered filled pyramid লিখুন।
4. Centered inverted pyramid লিখুন।
5. Size 1-সহ hollow square লিখুন।
6. Floyd's number triangle লিখুন।
7. Widest row একবার রেখে centered diamond লিখুন।
8. Hollow centered pyramid লিখুন।
9. Middle row-এর সংজ্ঞা দিয়ে butterfly pattern লিখুন।

10. Row বদলালেও counter চলতে থাকে এমন numeric triangle লিখুন।

File, error ও test · 81–90

1. Study goal UTF-8 text-এ save ও reload করুন।
2. একাধিক path স্বাধীনভাবে পড়ুন, প্রতিটি missing file জানান।
3. Unrelated data না মুছে নামতা text file-এ লিখুন।
4. `csv` module দিয়ে quoted field পড়ুন।
5. Record list JSON-এ রেখে load-এর সময় validate করুন।
6. Bare `except` ছাড়া non-numeric input সামলান।
7. Function-এর domain ভাঙলে exception raise করুন।
8. Boundary-সহ calculation `unittest` দিয়ে check করুন।
9. Temporary directory-তে file persistence test করুন।
10. Minimal example-এ bug ধরুন এবং regression test যোগ করুন।

Object ও project · 91–100

1. Independent instance state-সহ study session model করুন।
2. Validated property-তে invalid value reject দেখান।
3. Unsupported-type behavior-সহ vector addition করুন।
4. Library borrowing system-এ composition ও inheritance তুলনা করুন।
5. Calculation ও file I/O না মিশিয়ে calculator history দিন।
6. Validated JSON থেকে quiz question load করুন।
7. Expense tracker-এ monthly filter দিন।
8. ভেতরে exact minor unit রেখে expense CSV export করুন।
9. Auto-send ছাড়া assistant-এ draft-preview confirmation যোগ করুন।
10. README, contract, tests, sample data ও backup strategy-সহ capstone plan করুন।

30 · নির্বাচিত সমস্যার যুক্তিসহ সমাধান

আগে নিজে চেষ্টা করুন, তারপর তুলনা করুন। এই ১৫টি solution section reusable reasoning শেখায়; এগুলো একশটি challenge-এর পূর্ণ answer key নয়। প্রতিটি code standalone এবং companion folder-এ আছে।

A · Leap year · Challenge 15

Century year হলে 400 দিয়ে divisibility দরকার। অন্য ক্ষেত্রে 4 দিয়ে বিভাজ্য কিন্তু 100 দিয়ে নয়।

```
def is_leap_year(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)

for year in [1900, 2000, 2024, 2023]:
    print(year, is_leap_year(year))
```

Output: আলাদা line-এ `1900 False`, `2000 True`, `2024 True`, `2023 False`। এই exercise-এ domain positive Gregorian year।

B · Prime পরীক্ষা · Challenge 24

কোনো factor square root-এর বড় হলে তার paired factor ছোট হয়। তাই integer square root পর্যন্ত divisor পরীক্ষা যথেষ্ট। Range-এর stop বাদ যায় বলে `+1` দরকার।

```
from math import isqrt


def is_prime(number):
    if number < 2:
        return False
    for divisor in range(2, isqrt(number) + 1):
        if number % divisor == 0:
            return False
    return True

print([number for number in range(1, 21) if is_prime(number)])
```

Output: `[2, 3, 5, 7, 11, 13, 17, 19]`। Domain integer। 1 ও 49-এর মতো perfect square দিয়ে boundary bug ধরুন।

C · Fibonacci · Challenge 26

এখানে convention 0, 1, 1, 2, 3, ...। আগের pair একই সঙ্গে ব্যবহার করে next pair তৈরি হয়। Update-এর আগে append করায় প্রথম term 0।

```
def fibonacci(count):
    if count < 0:
        raise ValueError("count cannot be negative")
    first, second = 0, 1
    result = []
    for _ in range(count):
        result.append(first)
        first, second = second, first + second
    return result

print(fibonacci(8))
print(fibonacci(0))
```

Output: `[0, 1, 1, 2, 3, 5, 8, 13]`, তারপর `[]`। Count non-negative integer হতে হবে। Zero count-এ loop না চলেই empty list ফেরে।

D · GCD ও LCM · Challenges 27–28

Euclid-এ `(a, b)` বদলে `(b, a % b)` হয়; remainder zero হলে থামে। LCM-এ গুণের আগে ভাগ করলে intermediate value ছোট থাকে।

```
def gcd(a, b):
    a, b = abs(a), abs(b)
    while b:
        a, b = b, a % b
    return a


def lcm(a, b):
    if a == 0 or b == 0:
        return 0
    return abs((a // gcd(a, b)) * b)

print(gcd(48, 18))
print(lcm(12, 18))
print(lcm(0, 18))
```

Output: `6`, `36`, `0`। Integer domain। এই convention-এ `gcd(0, 0)` হলো 0 এবং কোনো input zero হলে LCM 0।

E · Sign রেখে digit reverse · Challenge 29

Sign আলাদা রাখুন। Absolute value থেকে শেষ digit নিন, result-এ ডানে যোগ করুন, original number-এর শেষ digit বাদ দিন। Integer result-এ leading zero থাকে না।

```
def reverse_integer(number):
    sign = -1 if number < 0 else 1
    remaining = abs(number)
    result = 0
    while remaining:
        result = result * 10 + remaining % 10
        remaining //= 10
    return sign * result

for number in [1200, -305, 0]:
    print(reverse_integer(number))
```

Output: `21`, `-503`, `0`। 1200 উল্টে 21, চার character-এর string নয়। Zero-তে loop না চললেও initial result সঠিক।

F · Normalized palindrome · Challenge 37

এই policy-তে non-alphanumeric বাদ এবং caseless comparison। এটি সাধারণ code-point exercise; পূর্ণ grapheme-aware ভাষাগত tool নয়।

```
def is_phrase_palindrome(text):
    cleaned = "".join(character for character in text.casefold()
                      if character.isalnum())
    return bool(cleaned) and cleaned == cleaned[::-1]

print(is_phrase_palindrome("Never odd or even"))
print(is_phrase_palindrome("Python"))
print(is_phrase_palindrome("!!!"))
```

Output: `True`, `False`, `False`। Normalization-এর পরে empty text ইচ্ছাকৃতভাবে reject করা হয়েছে। নইলে শুধু punctuation-ও palindrome বলে গণ্য হতে পারত।

G · Frequency tie নির্ধারিত রাখুন · Challenge 40

প্রথম appearance অনুযায়ী dictionary-তে count জমে। Equal maximum হলে `max()` প্রথম পাওয়া item রাখে। তাই tie policy reproducible।

```
def most_frequent_character(text):
    if not text:
        raise ValueError("text cannot be empty")
    counts = {}
    for character in text:
        counts[character] = counts.get(character, 0) + 1
    return max(counts, key=counts.get)

print(most_frequent_character("baba"))
```

Output: `b`। দুটিই দুইবার, কিন্তু `b` আগে এসেছে। `max(set(text), key=text.count)` দিয়ে first-appearance tie guarantee পাওয়া যায় না।

H · List chunk · Challenge 49

Positive step-এ start position নিন; slicing শেষের ছোট chunk-ও সামলে দেয়।

```
def chunks(values, size):
    if size <= 0:
        raise ValueError("size must be positive")
    return [values[start:start + size]
            for start in range(0, len(values), size)]

print(chunks([1, 2, 3, 4, 5], 2))
print(chunks([], 2))
```

Output: `[[1, 2], [3, 4], [5]]`, তারপর `[]`। Domain sliceable list ও positive integer size। Zero/negative size error, empty answer নয়।

I · Order রেখে unique · Challenge 48

`seen` set বলে item আগে এসেছিল কি না। Result list accepted item-এর order রাখে।

```
def unique_in_order(values):
    seen = set()
    result = []
    for value in values:
        if value not in seen:
            seen.add(value)
            result.append(value)
    return result

print(unique_in_order([3, 1, 3, 2, 1]))
```

Output: `[3, 1, 2]`। Item hashable হতে হবে। Unhashable record হলে stable key বা ভিন্ন equality-based পদ্ধতি লাগবে।

J · নিরাপদ default · Challenge 64

List দেওয়া না হলেই নতুন list তৈরি করুন। Caller list দিলে এই function ইচ্ছাকৃতভাবে সেটি বদলায়—এই behavior contract-এ বলা দরকার।

```
def add_tag(tag, tags=None):
    if tags is None:
        tags = []
    tags.append(tag)
    return tags

print(add_tag("python"))
print(add_tag("loops"))
```

Output: `['python']`, তারপর `['loops']`। দ্বিতীয় call প্রথম call-এর tag পায় না। `tags=[]` default হলে এই independence থাকত না।

K · Closure ও decorator · Challenges 67–68

Returned function enclosing `factor` মনে রাখে। Decorator function-কে wrapper দিয়ে বদলায় এবং `wraps` metadata রাখতে সাহায্য করে।

```
from functools import wraps


def multiplier(factor):
    def multiply(value):
        return value * factor
    return multiply


def announce(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(f"Calling {function.__name__}")
        return function(*args, **kwargs)
    return wrapper


@announce
def double(value):
    return multiplier(2)(value)

print(double(5))
```

Output: `Calling double`, তারপর `10`। `@announce`-এর ফলে `double` নাম এখন `announce`-এর ফেরানো wrapper নির্দেশ করে। Wrapper আসল function-এর result ফেরত দিচ্ছে—এটি বাদ দিলে caller `None` পেতে পারে।

L · Floyd's triangle · Challenge 76

Counter দুই loop-এর বাইরে। Inner loop শুধু row-তে কয়টি সংখ্যা থাকবে তা ঠিক করে; counter নতুন করে শুরু করে না।

```
number = 1
for row in range(1, 5):
    values = []
    for _ in range(row):
        values.append(str(number))
        number += 1
    print(" ".join(values))
```

```
1
2 3
4 5 6
7 8 9 10
```

M · Diamond · Challenge 77

Upper half-এর পরে lower half এক row ছোট থেকে শুরু। তাই widest row duplicate হয় না।

```
height = 3
rows = list(range(1, height + 1)) + list(range(height - 1, 0, -1))
for row in rows:
    print(" " * (height - row) + "*" * (2 * row - 1))
```

```
  *
 ***
*****
 ***
  *
```

N · Hollow pyramid · Challenge 78

প্রথম ও শেষ column sloping edge। শেষ row পুরো ভরে base হয়। অন্য cell space।

```
height = 4
for row in range(1, height + 1):
    width = 2 * row - 1
    cells = []
    for column in range(width):
        border = column in (0, width - 1) or row == height
        cells.append("*" if border else " ")
    print(" " * (height - row) + "".join(cells))
```

```
   *
  * *
 *   *
*******
```

O · Vector addition · Challenge 93

Dataclass coordinate স্পষ্ট করে। অন্য type হলে `NotImplemented` দিয়ে Python-এর operator protocol মানা হয়।

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Vector:
    x: float
    y: float

    def __add__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"


print(Vector(2, 3) + Vector(4, 1))
```

Output: `(6, 4)`। Type annotation numeric intent জানায়, runtime constructor validation নয়। Frozen dataclass সাধারণ field reassignment আটকায়, কিন্তু যেকোনো field-এর গভীর immutability নিশ্চিত করে না।

31 · Quick reference ও পরবর্তী পথ

কোন কাজে কোন tool

প্রয়োজন	আগে যা বিবেচনা করবেন
User-এর text	`input()`
ফল দেখানো	`print()`, f-string
পথ বাছাই	`if` / `elif` / `else`
Condition বদলানো পর্যন্ত repetition	`while`
Item ধরে কাজ	`for`
Counter ও item	`enumerate()`
Ordered editable collection	`list`
Fixed group	`tuple`
Key দিয়ে lookup	`dict`
Unique item, membership	`set`
Reusable logic	`def`, `return`
File path	`pathlib.Path`
Structured local text data	`json`
Tabular text	`csv`
Expected error recovery	নির্দিষ্ট `except`
Automated check	`unittest`
আলাদা dependency	`venv`, `python -m pip`

Debugging checklist

- সঠিক file save ও run করেছি?
- Interpreter ও working folder ঠিক?
- Failing line-এ actual type ও value কী?

- Stop boundary inclusive নাকি exclusive?
- Input empty, zero, negative বা duplicate হতে পারে?
- Result return করছি, নাকি শুধু print?
- অজান্তে mutable object share করছি?
- প্রয়োজনীয় bug কোনো broad exception দিয়ে লুকিয়েছি?
- Result set order বা randomness-এর ওপর নির্ভর করছে?
- আরও ছোট code-এ সমস্যাটি দেখানো যায়?

সংক্ষিপ্ত শব্দকোষ

Algorithm: নির্ধারিত সমস্যা সমাধানের ধাপ।
Argument: call-এর সময় function-কে দেওয়া মান।
Binding: নাম ও object-এর সম্পর্ক।
Boolean: সত্য/মিথ্যা value।
Contract: accepted input, result ও failure behavior-এর নিয়ম।
Exception: কোনো operation স্বাভাবিকভাবে এগোতে না পারার সংকেতবাহী object।
Hashable: consistent hash/equality behavior-সহ hash-based lookup-এর উপযোগী।
Immutable: object-এর নিজের level-এ in-place পরিবর্তনযোগ্য নয়।
Instance: class-এর নির্দিষ্ট object।
Iterable: যেখান থেকে iterator item নিতে পারে।
Iterator: একবারে একটি করে item দেয়, শেষে exhaust হয়।
Mutable: in-place বদলানো যায়।
Parameter: function definition-এ argument গ্রহণের নাম।
Persistence: process শেষেও data রাখা।
Recursion: একই সমস্যার ছোট instance-এর call দিয়ে সমাধান।
Scope: নাম খোঁজার context বা সীমানা।
Trace: execution-এর ধাপ ও পরিবর্তিত মানের record।
Validation: data কাজটির নিয়ম মানে কি না পরীক্ষা।

চার সপ্তাহের নমনীয় পরিকল্পনা

সপ্তাহ ১: ১–৮ অধ্যায়। প্রতিটি output অনুমান করুন, code চালান, foundation ও loop challenge করুন। ভুলের খাতা রাখুন।

সপ্তাহ ২: ৯–১৬ অধ্যায়। Pattern আঁকুন, string ও collection বদলান, repeated logic function-এ নিন। তাড়াহুড়া করে সব না করে দশটি উপযোগী challenge ভালোভাবে করুন।

সপ্তাহ ৩: ১৭–২৪ অধ্যায়। Venv, validated file data, traceback ও test। Class বানানোর আগে কেন দরকার ব্যাখ্যা করুন।

সপ্তাহ ৪: একবারে একটি project। একটি feature, একটি boundary test ও README-তে একটি উন্নতি যোগ করুন। এক পাতায় requirement লেখা যায় এমন capstone বাছুন।

এটি deadline নয়। আটকে গেলে ছোট উদাহরণে ফিরুন। না বুঝে পঞ্চাশ line copy করার চেয়ে পাঁচ line গভীরভাবে বোঝা বেশি মূল্যবান।

নির্ভরযোগ্য reference

ভাষার behavior ও standard-library API-এর মূল reference Python documentation। এগুলো online resource; বই পড়া বা standard-library project চালাতে network বাধ্যতামূলক নয়।

- Python tutorial
- Built-in functions
- Standard library
- Virtual environment
- Unittest
- PEP 8 style guide

বিষয় নির্বাচনে সরবরাহ করা beginner notes, program collection, pattern example, input/output slide ও CodeWithHarry handbook excerpt সহায়ক ছিল। এই সংস্করণের ব্যাখ্যা, বিন্যাস, worked example ও project নতুনভাবে লেখা; screenshot page দিয়ে বই বানানো হয়নি। মূল reference উপকরণ তাদের নিজ নিজ নির্মাতার; সেগুলোকে MaksDev Studio-এর মূল প্রকাশনা হিসেবে দাবি করা হচ্ছে না।

এবার নিজের কিছু বানান

শুরু হয়েছিল পর্দায় একটি ফল দেখানো দিয়ে। এখন আপনি সমস্যাকে পরিকল্পনায়, পরিকল্পনাকে code-এ এবং code-কে test করা যায় এমন program-এ রূপ দেওয়ার ধারণা জানেন।

সব feature শেখা পর্যন্ত অপেক্ষা করবেন না। নিজের কাজে লাগে এমন ছোট সমস্যা নিন। Input লিখুন, সহজ সঠিক version বানান, একটি awkward case test করুন এবং সিদ্ধান্তের কারণ ব্যাখ্যা করুন। তারপর পরের version বানান।

যুক্তি বুঝুন। কোড লিখুন। কাজে লাগে এমন কিছু বানান।

Md. Makshedul Islam
Student, EXIM Bank Agricultural University Bangladesh.

MaksDev Studio

- GitHub: MaksBloxX
- Repository: Python Project Lab
- LinkedIn: Md. Makshedul Islam